

Rapport de Stage de Fin d'Études

Pour l'obtention du Diplôme d'Ingénieur d'État et du Master

Conception et mise en œuvre d'un pipeline automatisé de contrôle de la qualité des données : Cas des données CSP et KYC

Filière :

Réalisé par :

SAAD BENDAHO

MIAGE (Méthodes Informatiques
Appliquées à la Gestion des
Entreprises)

MIAGE I2A (Informatique et
Intelligence Artificielle)

Encadrant de l'École :

Mr. Marouan Skiti

Encadrants de CIH BANK :

Mr. Mohamed Agouzzal

Mr. Marouan Naoui

Année Universitaire : 2025 – 2026

Dédicaces

Je dédie ce modeste travail :

À mes chers parents,

Aucun mot, aucune dédicace ne saurait exprimer mon amour, mon respect et ma profonde gratitude pour vos sacrifices, votre soutien inconditionnel et vos prières qui m'ont toujours accompagné tout au long de mon parcours universitaire. Que Dieu vous préserve et vous procure santé et longue vie.

À mes frères et sœurs,

Pour votre présence constante, vos encouragements et votre affection fraternelle.

À toute ma famille, mes proches et mes amis,

Qui ont toujours cru en moi et m'ont soutenu durant les moments les plus intenses de mes études.

Merci d'être toujours là pour moi.

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à Dieu le Tout-Puissant pour m'avoir donné la force, la patience et la persévérance nécessaires pour mener à bien ce travail.

Je tiens à exprimer mes sincères remerciements à mon école d'accueil, l'Université de Nice Sophia Antipolis (UNICA) et l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour m'avoir assuré une excellente formation académique et des bases solides pour entamer ma vie professionnelle.

J'adresse mes remerciements les plus chaleureux à mon encadrant pédagogique, **Mr. Marouan Skiti**, pour ses conseils méthodologiques avisés, sa disponibilité continue et la bienveillance dont il a fait preuve tout au long du suivi de ce projet de fin d'études.

Mes remerciements les plus vifs s'adressent à l'institution de mon stage, **CIH BANK**, pour m'avoir ouvert ses portes et permis de réaliser ce travail dans un environnement professionnel stimulant et formateur.

J'exprime toute ma reconnaissance à mes encadrants de stage chez CIH Bank, **Mr. Mohamed Agouzzal** (CDO - Chef du Data Office) et **Mr. Marouan Naoui** (Data Quality Analyst / Spécialiste Data Management & Gouvernance), pour leur accueil chaleureux, leur confiance, leur précieux encadrement et leur soutien technique rigoureux durant toute la durée de mon projet. Leurs directives éclairées m'ont grandement aidé à surmonter les difficultés techniques et à concevoir ce pipeline.

Enfin, je remercie l'ensemble des collaborateurs de CIH Bank et les membres de l'équipe Data Office pour leur esprit d'équipe, leur soutien et les échanges enrichissants que nous avons partagés. Que tous ceux qui ont contribué de près ou de loin à la réussite de ce projet trouvent ici l'expression de ma profonde gratitude.

Résumé

Dans un contexte de transformation digitale du secteur bancaire, la donnée constitue un levier stratégique essentiel pour la prise de décision, la gestion des risques et l'amélioration de la performance globale. Cependant, la qualité des données représente un enjeu majeur, car des données incomplètes, incohérentes ou erronées peuvent impacter significativement les processus métiers et les analyses.

Ce projet s'inscrit dans le cadre des initiatives de Data Governance au sein de CIH Bank. Il vise à concevoir et mettre en œuvre un pipeline automatisé de contrôle de la qualité des données, permettant d'assurer la fiabilité, la traçabilité et la cohérence des informations exploitées.

La solution proposée repose sur le traitement et le contrôle de données issues des systèmes opérationnels pour la base des tiers (qui alimentent le Data Lake) ainsi que de fichiers structurés, en intégrant des règles de contrôle couvrant plusieurs dimensions de la qualité des données telles que la complétude, la validité, l'unicité et l'intégrité.

L'automatisation des contrôles permet d'instaurer une surveillance continue de la qualité des données, avec la production d'indicateurs (KPI) facilitant le pilotage et la prise de décision. Ce projet contribue ainsi à renforcer la gouvernance des données et à soutenir les initiatives analytiques et décisionnelles de la banque.

Abstract

In the context of digital transformation within the banking sector, data has become a key strategic asset for decision-making, risk management, and overall performance improvement. However, data quality remains a major challenge, as incomplete, inconsistent, or erroneous data can significantly impact business processes and analytical outcomes.

This project is part of the Data Governance initiatives at CIH Bank. Its objective is to design and implement an automated data quality control pipeline to ensure the reliability, traceability, and consistency of the data being used.

The proposed solution is based on processing and controlling data from operational systems for the third-party database (which feed the Data Lake) as well as structured files, while applying a set of rules covering key data quality dimensions such as completeness, validity, uniqueness, and integrity.

By automating these controls, the solution enables continuous monitoring of data quality and the generation of key performance indicators (KPIs), supporting efficient data governance and informed decision-making. This project therefore contributes to strengthening the organization's data management framework and enhancing its analytical capabilities.

Table des matières

Dédicaces	i
Remerciements	ii
Résumé	iii
Abstract	iv
Liste des abréviations	ix
Introduction Générale	1
1 Présentation du cadre de projet	2
1.1 Présentation de l'organisme d'accueil	2
1.1.1 Missions de CIH BANK	2
1.1.2 Secteurs d'activité	3
1.1.3 Filiales	3
1.1.4 Actionnariat	3
1.1.5 Chiffres clés	3
1.1.6 Organigramme	4
1.1.7 Data Office	5
1.2 Étude de l'existant	6
1.2.1 Description de l'existant	6
1.2.2 Critique de l'existant	8
1.2.3 Solution proposée	8
1.3 Choix du modèle de développement	9
1.4 Planning prévisionnel	9
2 Analyse des Besoins et Spécification	11
2.1 Besoins fonctionnels	11
2.1.1 Automatisation et planification des contrôles	11
2.1.2 Configuration déclarative des règles de qualité	11
2.1.3 Historisation et traçabilité des indicateurs	12
2.1.4 Restitution visuelle et aide à la décision	12
2.2 Besoins non fonctionnels	12
2.2.1 Performance	12
2.2.2 Disponibilité	13
2.2.3 Maintenabilité	13
2.2.4 Sécurité	13
2.2.5 Conformité Réglementaire et Normative	13
2.3 Modélisation des cas d'utilisation	14

2.3.1	Acteurs du système	14
2.3.2	Diagramme global des cas d'utilisation	14
2.3.3	Description textuelle des cas d'utilisation principaux	15
3	Conception de la Solution	17
3.1	Modélisation dynamique	17
3.1.1	Diagramme de séquences	17
3.1.2	Diagramme d'activités	18
3.2	Modélisation statique	19
3.2.1	Modélisation de la table de faits	19
3.2.2	Modèle relationnel	21
3.2.3	Dictionnaire de données	21
3.2.4	Catalogue des règles de qualité	21
3.3	Architecture de l'application	24
3.3.1	Architecture logicielle (diagramme de composants)	24
3.3.2	Architecture matérielle (diagramme de déploiement)	26
3.3.3	Technologies de l'écosystème utilisées	27
4	Réalisation de la Solution	30
4.1	Environnement technique de développement	30
4.1.1	Environnement Hadoop et accès distant	30
4.1.2	Installation hors-ligne de Soda Core	31
4.2	Implémentation du moteur DQ — Soda Core et Spark	32
4.2.1	Arborescence du projet DQ	32
4.2.2	Principe de fonctionnement de Soda Core avec Spark	32
4.2.3	Fichier de configuration YAML (<code>checks_tiers.yml</code> — exemplaire)	33
4.2.4	Traduction des règles SodaCL en Spark SQL	34
4.2.5	Script Python principal (<code>dq_tiers_checks.py</code>)	35
4.3	Création des tables Hive physiques de persistance	38
4.4	Création du DAG Airflow indépendant	38
4.5	Exposition et virtualisation sous Dremio	39
4.6	Réalisation du dashboard Tableau	40
4.6.1	Connexion Tableau à Dremio et traitement analytique	40
4.6.2	Onglet 1 — Vue d'ensemble (Dashboard Global)	40
4.6.3	Onglet 2 — Heatmap Qualité	41
4.6.4	Onglet 3 — Top Anomalies	41
	Conclusion générale	43
	Bibliographie et Nétographie	44

Table des figures

1	Chiffres clés de CIH BANK (31/12/2024)	4
2	Organigramme de CIH BANK	5
3	Architecture matérielle Big Data	7
4	Planning prévisionnel du projet	10
5	Diagramme global des cas d'utilisation de la solution DQ Tiers	14
6	Diagramme de séquences du pipeline de contrôle et restitution DQ	18
7	Diagramme d'activités du workflow de remédiation DQ	18
8	Modélisation de la grande table de faits <code>fact_tiers_dq_consolidated</code> et ses 3 tables sources	20
9	Diagramme de composants logiciels de la solution DQ	25
10	Architecture matérielle et déploiement dans le cluster Big Data	27
11	Interface MobaXterm utilisée pour l'accès SSH au nœud Edge du cluster Cloudera	31
12	Lignage des données dans Dremio montrant la création de la vue sémantique à partir des 3 tables Hive	39
13	Dashboard Tableau — Vue d'ensemble de la qualité des données	40
14	Dashboard Tableau — Heatmap Qualité par colonne et par date	41
15	Dashboard Tableau — Top anomalies et colonnes en échec	42

Liste des tableaux

2	Informations générales sur le groupe CIH BANK [10]	2
3	Répartition du capital de CIH BANK [4]	3
4	Acteurs du système de Data Quality	14
5	Description du cas d'utilisation 01	15
6	Description du cas d'utilisation 02	15
7	Description du cas d'utilisation 03	16
8	Dictionnaire simplifié des tables de faits de Qualité des Données	21
9	Dimension Complétude — Données Tiers (Référentiel Tiers)	22
10	Dimension Conformité — Données Tiers (Référentiel Tiers)	23
11	Dimension Cohérence — Données Tiers (Référentiel Tiers)	23
12	Dimension Unicité — Données Tiers (Référentiel Tiers)	24
13	Dimension Actualité et Pipeline — Flux Batch	24
14	Versions des composants de l'environnement technique	31

Liste des abréviations

Abréviation	Signification
ANRF	Autorité Nationale du Renseignement Financier
API	Application Programming Interface
BCBS	Basel Committee on Banking Supervision
BI	Business Intelligence
CDO	Chief Data Officer
CIH	Crédit Immobilier et Hôtelier
CNDP	Commission Nationale de contrôle de la Protection des Données à caractère personnel
CNSS	Caisse Nationale de Sécurité Sociale
CSP	Catégories Socio-Professionnelles
CSV	Comma-Separated Values
DQ	Data Quality
DWH	Data Warehouse
ETL	Extract, Transform, Load
ISO	Organisation Internationale de Normalisation
KPI	Key Performance Indicator
KYC	Know Your Customer
LLM	Large Language Model
RGPD	Règlement Général sur la Protection des Données
SQL	Structured Query Language

Introduction Générale

Au cours des dernières années, le secteur bancaire a connu une transformation profonde sous l'impulsion de la digitalisation et de l'essor des technologies liées à la donnée. Dans ce contexte, les institutions financières s'appuient de plus en plus sur leurs systèmes d'information pour analyser les comportements clients, optimiser leurs processus et piloter leurs activités de manière efficace.

La donnée est ainsi devenue un actif stratégique. Toutefois, sa valeur dépend directement de sa qualité. Des données inexactes ou mal structurées peuvent engendrer des erreurs d'analyse, des décisions inadaptées et des risques importants, notamment dans des domaines sensibles tels que l'octroi de crédit ou la conformité réglementaire.

Face à ces enjeux, les organisations mettent en place des démarches de Data Governance visant à structurer, contrôler et valoriser leurs données. Parmi les axes majeurs de cette gouvernance figure la gestion de la qualité des données, qui repose sur la définition de règles de contrôle, leur automatisation et le suivi d'indicateurs de performance.

C'est dans ce cadre que s'inscrit le présent projet, réalisé au sein de CIH Bank. Il a pour objectif de concevoir et de mettre en œuvre une solution automatisée permettant de contrôler, mesurer et améliorer la qualité des données.

Le présent rapport est structuré en quatre chapitres : le premier chapitre présente l'organisme d'accueil ainsi que l'étude de l'existant, le deuxième expose l'analyse des besoins fonctionnels et non fonctionnels, le troisième décrit la conception théorique et l'architecture de la solution, et enfin, le quatrième détaille son implémentation technique et les résultats obtenus.

Chapitre 1

Présentation du cadre de projet

Introduction du chapitre

Dans ce chapitre, nous présenterons une description générale de l'organisme d'accueil, CIH BANK, d'un point de vue organisationnel. Nous aborderons ensuite l'étude de l'existant, le choix du modèle de développement ainsi que le planning prévisionnel du projet, afin de mieux contextualiser notre travail de fin d'études.

1.1 Présentation de l'organisme d'accueil

Le Crédit Immobilier et Hôtelier (CIH), plus communément appelé CIH BANK, est une banque marocaine dont le siège social se trouve à Casablanca. Historiquement connu pour son activité dans le secteur de l'immobilier, le CIH propose désormais des services bancaires variés à ses clients, particuliers et professionnels. [10]

CIH BANK a été fondée en 1920 sous le nom de Caisse de Prêts Immobiliers du Maroc (CPIM). En 1967, le groupe a étendu son activité au secteur hôtelier et a changé de nom pour devenir le Crédit Immobilier et Hôtelier. [10]

Création	1920
Fondateur	L'État marocain
Introduction en bourse	1967
Forme juridique	Société anonyme
Slogan	La banque de demain dès aujourd'hui
Siège social	187, Avenue Hassan II, Casablanca, Maroc
Président Directeur Général (PDG)	Lotfi SEKKAT
Société mère	Caisse de Dépôt et de Gestion
Effectif	2 181
Site web	www.cihbank.ma

TABLE 2 – Informations générales sur le groupe CIH BANK [10]

1.1.1 Missions de CIH BANK

Spécialisé dans le financement immobilier et hôtelier, le CIH est une banque à vocation particulière :

- Elle est un partenaire des pouvoirs publics en matière de financement du logement depuis 1920 ,

- Elle finance les promoteurs immobiliers au Maroc, par deux canaux différents :
 - En amont : Par le financement des promoteurs publics et privés (acquisition et viabilisation des terrains ainsi que la construction des programmes immobiliers)
 - En aval : Par l’octroi de crédits au logement pour les particuliers acquéreurs, sans oublier les services et produits proposés par la banque pour sa clientèle.

1.1.2 Secteurs d’activité

Le groupe CIH BANK et ses filiales offrent une vaste gamme de produits et de services à ses clients, particuliers et professionnels :

- Services bancaires ,
- Prêts aux particuliers (prêts immobiliers, crédits à la consommation, etc.) ,
- Financement d’entreprises (prêts d’investissement et d’exploitation, crédit-bail, etc.) ,
- Assurance ,
- Salle des marchés.

1.1.3 Filiales

Le groupe CIH BANK est constitué de plusieurs filiales :

- SOFAC : entreprise spécialisée dans les solutions de crédit ,
- SOFASSUR : entreprise spécialisée dans les services d’assurance ,
- Maroc Leasing : entreprise de leasing qui propose des solutions de financement pour les véhicules, le BTP, l’immobilier et la santé [6] ,
- CIH Courtage : courtier en assurance ,
- Umnia Bank : banque participative ,
- AJAR INVEST : société de gestion des Organismes de Placement Collectif Immobilier (OPCI), dont CIH BANK détient 40 % des parts [1] .

1.1.4 Actionnariat

La répartition du capital du groupe CIH BANK est présentée dans le tableau 3 :

TABLE 3 – Répartition du capital de CIH BANK [4]

Actionnaire	Part
Massira Capital Management	55,66 %
Flottant en bourse	16,37 %
Atlanta Sanad	11,61 %
Caisse de Dépôt et de Gestion (CDG)	6,69 %
Régime Collectif d’Allocation de Retraite (RCAR)	5,21 %
Personnel de la banque	4,36 %
Groupe Holmarcom	0,12 %

1.1.5 Chiffres clés

La figure 1 présente les chiffres clés de CIH BANK.

Les chiffres clés de CIH BANK reflètent la situation financière et opérationnelle de l’institution. Avec un capital social de 3,149 milliards de dirhams et un total bilan de 141

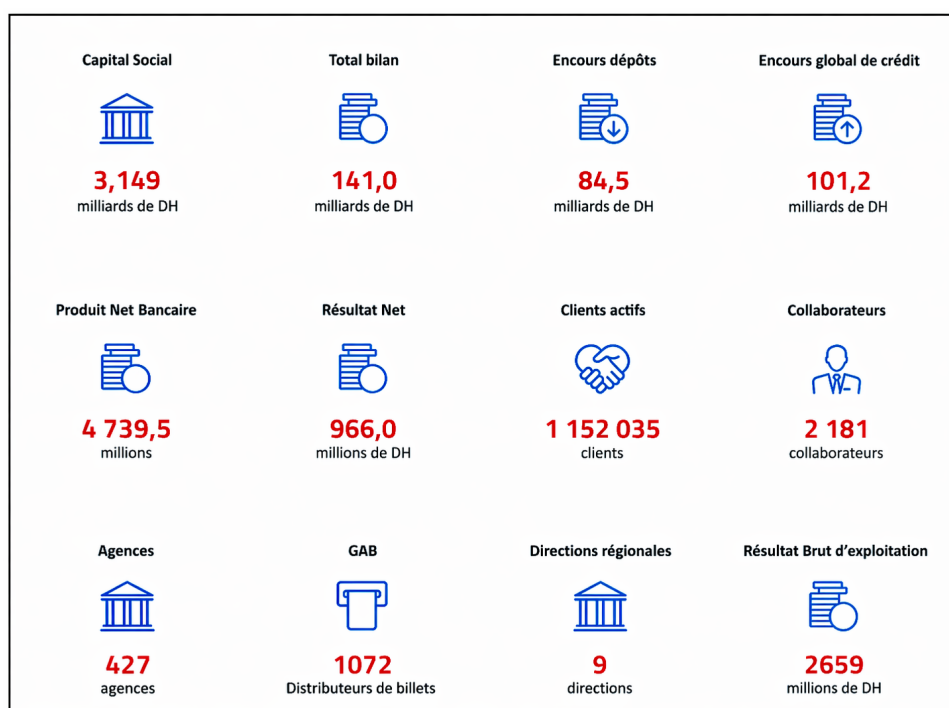


FIGURE 1 – Chiffres clés de CIH BANK (31/12/2024)

milliards de dirhams, la banque dispose d'une base financière significative. L'encours des dépôts s'élève à 84,5 milliards de dirhams, tandis que l'encours global de crédit atteint 101,2 milliards de dirhams. Le produit net bancaire est de 4 739,5 millions de dirhams, avec un résultat net de 966 millions de dirhams et un résultat brut d'exploitation de 2 659 millions de dirhams. La banque compte plus de 1,15 million de clients actifs, 2 181 collaborateurs, 427 agences, 1 072 guichets automatiques et 9 directions régionales. Ces indicateurs fournissent une vue d'ensemble de l'activité, de la structure et des résultats financiers de l'institution.

1.1.6 Organigramme

Structure hiérarchique de CIH BANK

La structure hiérarchique de CIH BANK s'articule autour d'une direction générale pilotée par le Président Directeur Général, appuyée par des fonctions de support stratégique et de contrôle (Audit, Conformité, Risques, Capital Humain, etc.) ainsi que diverses fonctions rattachées. L'activité commerciale et opérationnelle de la banque est quant à elle répartie en plusieurs pôles métiers majeurs : la Banque des Particuliers et Professionnels, la Banque de l'Entreprise et de l'Immobilier, la Banque de l'Investissement, le Financement & Recouvrement, et les Services à la Clientèle & Réseaux.

Parmi ces différentes directions, notre attention se porte plus particulièrement sur l'entité technologique qui encadre notre organisme d'accueil :

Fonctions technologiques et data

- Services Technologiques et Opérations : D. BENNOUNA ,
- Sécurité des Systèmes d'Information ,
- Qualité ,
- Pôle Systèmes d'Information : T. CHARKAOUI ,
- Data Management & Intelligence Artificielle :

— Data Office.

L'organigramme global de CIH BANK est illustré dans la figure 2 ci-dessous.

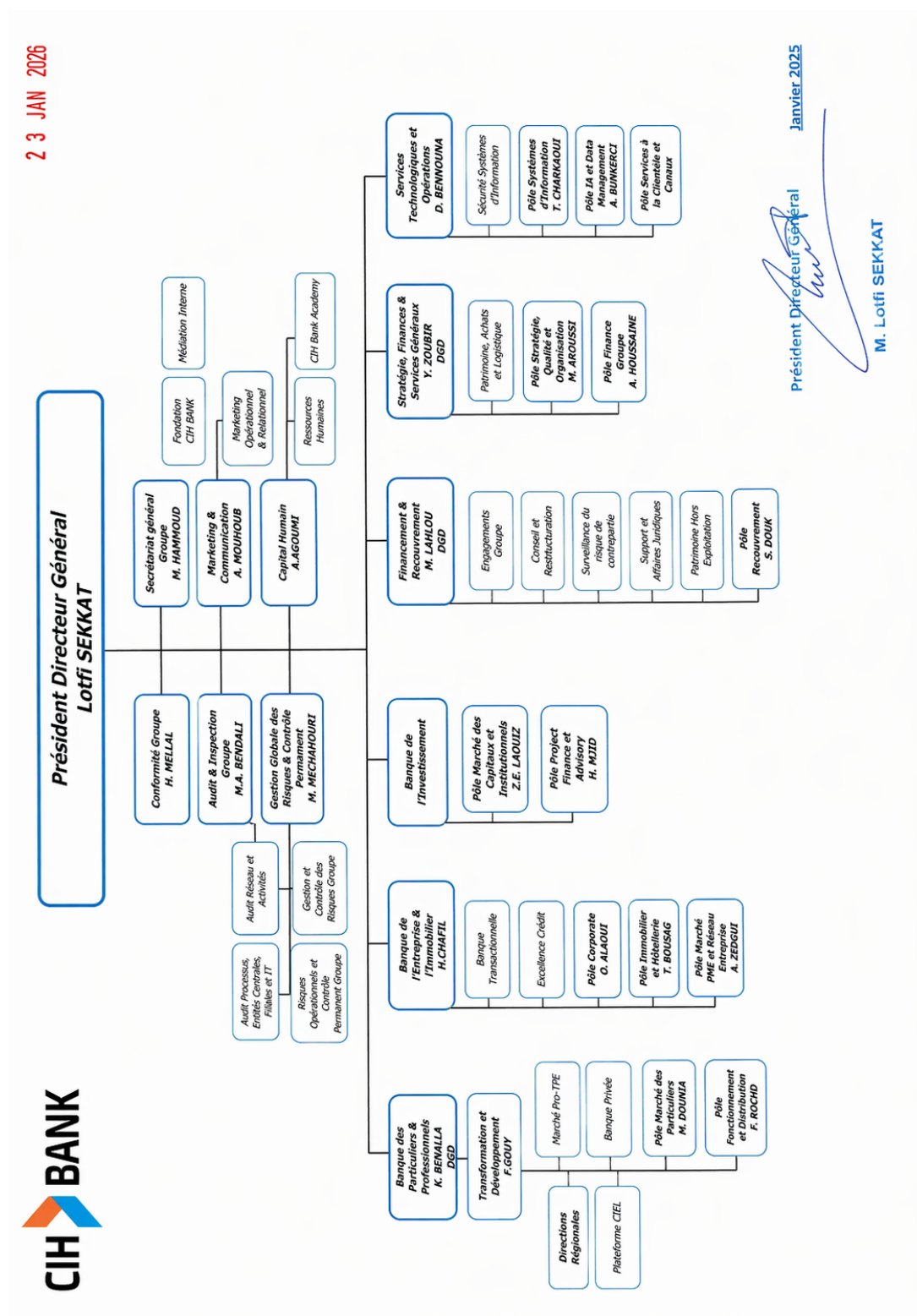


FIGURE 2 – Organigramme de CIH BANK

1.1.7 Data Office

Dans le cadre de notre stage, nous avons intégré le Data Office de CIH BANK, une entité dédiée à la gestion, au traitement et à la valorisation des données de la banque.

Le Data Office est aujourd’hui rattaché au pôle Services Technologiques et Opérations, plus précisément au sein de la direction Data Management et Intelligence Artificielle. Cette organisation reflète l’importance stratégique accordée à la donnée dans le pilotage des activités de la banque.

Cette entité regroupe plusieurs profils complémentaires intervenant sur l’ensemble du cycle de vie de la donnée, notamment :

- Les Data Engineers, en charge de l’ingestion, du traitement et de la mise à disposition des données,
- Les Data Analysts, responsables de l’analyse des données et de la production d’indicateurs décisionnels,
- Les Data Scientists, spécialisés dans la modélisation avancée et l’analyse prédictive,
- L’équipe Data Management, qui assure la gouvernance, la qualité et la structuration des données.

Les données sont collectées à partir des différents systèmes opérationnels de la banque, puis intégrées et centralisées au sein d’un Data Lake. Elles sont ensuite transformées et mises à disposition pour répondre aux besoins analytiques, réglementaires et métiers.

Dans ce cadre, le Data Office joue un rôle central dans la mise en place des pratiques de gestion de la qualité des données.

1.2 Étude de l’existant

1.2.1 Description de l’existant

Contexte et enjeux de la qualité des données bancaires

Dans l’écosystème financier contemporain, la transition vers une économie numérique a redéfini la nature des institutions bancaires. La donnée n’est plus un simple sous-produit de l’activité transactionnelle, mais s’impose comme une infrastructure essentielle sur laquelle reposent l’évaluation des risques, la conformité réglementaire (notamment BCBS 239 et le guide RDARR de la BCE) et les décisions stratégiques. Sur le plan économique, le coût de la non-qualité est très élevé. La « règle de dix » (*Rule of Ten*) stipule qu’il coûte dix fois plus cher de corriger une donnée en aval que de la valider directement à la source. Une gouvernance proactive apporte une valeur mesurable : optimisation des fonds propres, efficacité opérationnelle et sécurisation des décisions basées sur l’intelligence artificielle.

Le domaine Tiers : Cartographie et données

Historiquement, la vérification de la qualité des données au sein de CIH BANK était un processus réactif déclenché sur demande, ce qui a motivé la création de l’entité Data Management (DM). Notre projet s’inscrit dans cette volonté d’amélioration continue et cible plus particulièrement le **Référentiel Tiers**. Ce référentiel regroupe l’ensemble des informations relatives aux personnes physiques et morales en relation avec la banque.

Dans le cadre de l’ingestion des données vers le Data Lake, l’équipe travaille sur plus d’une dizaine de tables du domaine Tiers. À titre d’exemple et d’illustration, voici les cinq tables principales extraites du système opérant Oracle NOVA (CCM) qui structurent notre périmètre de base :

- **identity** : Données relatives aux pièces d’identité (type, numéro, date d’expiration),
- **legalentity** : Informations sur les personnes morales (raison sociale, forme juridique),
- **person** : Informations générales sur le statut du client (marché, agence, type),

- **professionaldescription** : Caractéristiques professionnelles et financières des entités,
- **np_description (Natural Person Description)** : Données d'état civil, d'identification et informations KYC (Know Your Customer) pour les personnes physiques.

Bien que le pipeline automatisé de qualité de données déployé couvre l'ensemble du domaine Tiers (soit une quinzaine de tables issues de NOVA), **pour des soucis de clarté et d'illustration, les implémentations techniques et les règles de Data Quality présentées dans la suite de ce rapport se focaliseront particulièrement sur les tables np_description et jobs**. Ces tables sont éminemment stratégiques : np_description concentre les données KYC indispensables à l'identification réglementaire du client (nom, prénom, date de naissance, genre, nationalité, etc.), tandis que jobs contient des informations essentielles pour la catégorisation socio-professionnelle (CSP).

Architecture et pipeline de données

L'ensemble des données ingérées au sein de CIH BANK, incluant celles du domaine Tiers, transite par une plateforme robuste centralisée s'appuyant sur une infrastructure Big Data de pointe. Tout l'écosystème analytique repose sur ce système unifié pour le traitement et la valorisation de la donnée.

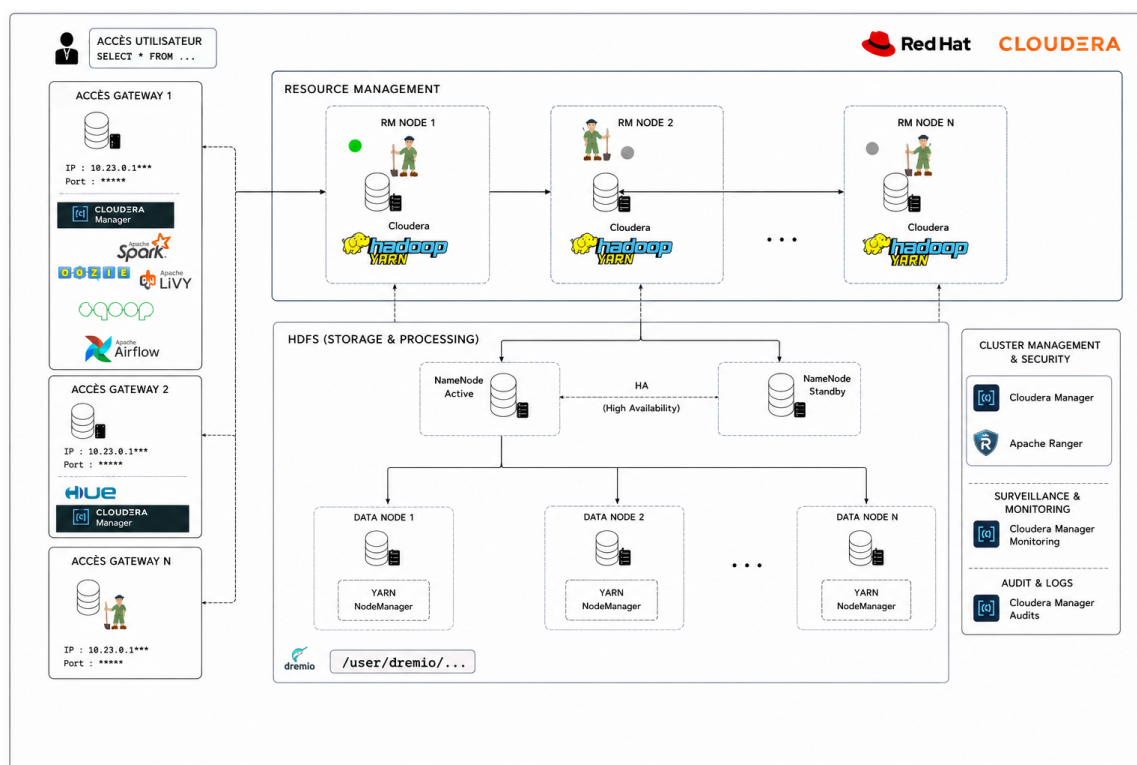


FIGURE 3 – Architecture matérielle Big Data

L'architecture matérielle et logicielle de cette plateforme de production s'articule autour d'un **cluster Cloudera et Dremio** composé de 8 nœuds (3 Masters, 3 Workers, 2 Gateways). Le flux global des données, de la source à l'exploitation, suit un cycle de vie standardisé en plusieurs étapes :

1. **Ingestion** : Les données brutes issues des systèmes opérants (tels qu'Oracle NOVA) sont ingérées massivement (souvent via Sqoop ou Kafka) et stockées dans la zone RAW du HDFS.

2. **Transformation** : Des jobs de traitement distribué (généralement via Apache Spark) nettoient, enrichissent et modélisent ces données brutes pour les charger dans la zone de STAGING.
3. **Virtualisation et Métadonnées** : Le moteur de données Dremio se connecte aux catalogues (Hive Metastore) pour virtualiser les tables. Il offre une couche sémantique puissante permettant de croiser et d'interroger les données sans les dupliquer.
4. **Exposition et Datamarts** : Les données transformées sont consolidées et exposées sous forme de Datamarts (ou vues métiers spécifiques). Ces ensembles de données prêts à l'emploi alimentent les outils de restitution (comme Tableau) pour répondre aux besoins analytiques des métiers.

1.2.2 Critique de l'existant

Bien que cette infrastructure de production Cloudera soit robuste, l'absence d'un contrôle automatisé de la qualité des données en amont pose plusieurs problèmes majeurs transverses à l'ensemble des tables du domaine Tiers. Pour illustrer l'impact de ces problèmes, nous prendrons l'exemple des tables `np_description` et `jobs`, qui contiennent des données KYC et CSP particulièrement sensibles concernant les personnes physiques :

- **Absence de validation à la source** : Les données transitent vers Dremio sans aucune validation intermédiaire. Des incohérences flagrantes persistent, comme des discordances entre le genre et la civilité du client, ou des dates de naissance remplies par défaut (ex : 01/01/XXXX) ,
- **Risques de non-conformité** : L'absence ou l'invalidité de certains champs critiques, tels que l'identifiant FATCA pour les clients concernés, expose la banque à des risques réglementaires et à d'éventuelles sanctions ,
- **Démarche réactive et manuelle** : Lorsqu'une anomalie est repérée par les métiers en bout de chaîne, la réconciliation implique asynchronement plusieurs équipes pour des corrections curatives chronophages, créant ainsi des processus inefficaces ,
- **Manque de traçabilité** : Aucune métrique n'est historisée. Les Data Stewards ne disposent d'aucun outil pour prioriser ou surveiller la santé des données KYC au quotidien.

1.2.3 Solution proposée

Pour pallier ces limites, nous proposons de concevoir et d'implémenter un **pipeline automatisé de contrôle de la qualité des données** couvrant l'ensemble du périmètre Tiers, intégré directement dans la zone RAW de l'infrastructure Hadoop.

La solution technique s'articulera autour des axes suivants :

- **Automatisation via Soda Core** : L'intégration de la bibliothèque Soda Core au sein des workflows Airflow existants permettra de valider systématiquement les données KYC (exhaustivité, unicité, validité de format, cohérence croisée) dès leur ingestion ,
- **Approche proactive** : En signalant les anomalies en zone RAW, nous empêchons l'exploitation de données erronées par les métiers, appliquant ainsi le principe de la *Rule of Ten* ,
- **Monitoring et Restitution** : Les résultats des contrôles et les anomalies seront persistés dans des tables Hive dédiées. Un tableau de bord interactif sera ensuite construit pour offrir aux Data Stewards une vue claire de la santé des tables

du référentiel. Les données sensibles affichées dans les tableaux de bord seront systématiquement anonymisées ou masquées pour garantir la confidentialité et la sécurité des informations clients.

Cette solution permettra d'améliorer considérablement la gouvernance des données du domaine Tiers tout en optimisant le temps de traitement des anomalies.

1.3 Choix du modèle de développement

Dans le cadre de ce projet de fin d'études, plutôt que d'adopter un modèle purement séquentiel en cascade, nous avons opté pour une approche **itérative et incrémentale** s'inspirant des méthodes agiles.

Ce choix a été motivé par la complexité de l'écosystème Big Data existant et l'étendue du domaine Tiers. Il était impossible de livrer une solution parfaite en une seule fois. Le projet a donc été rythmé par la réalisation de *Proof of Concepts* (POC) et de *Minimum Viable Products* (MVP), permettant de valider les règles de qualité table par table, et étape par étape.

Cette démarche nous a permis de maintenir une collaboration continue avec les équipes métiers, et tout particulièrement avec le Data Steward du domaine Tiers et les équipes du Système d'Information (SI), via des points de synchronisation réguliers tout au long des six mois de stage.

1.4 Planning prévisionnel

Afin de mener à bien ce projet au sein du Data Office de CIH BANK, nous avons défini un planning de travail couvrant une période de six mois, du **9 février au 9 août 2026**.

Contrairement à un projet classique, l'avancement n'a pas été strictement linéaire. Les deux premiers mois ont été presque exclusivement dédiés à la compréhension fonctionnelle, technique, et à la documentation détaillée de l'architecture existante. Par la suite, les phases de conception, de réalisation des POCs, et de tests se sont superposées de manière itérative, accompagnées de réunions continues avec les métiers.

La figure 4 ci-dessous illustre ce déroulement sous forme d'un diagramme de Gantt, mettant en évidence la phase initiale de documentation, le cycle itératif de développement, et l'implication constante des acteurs métiers.

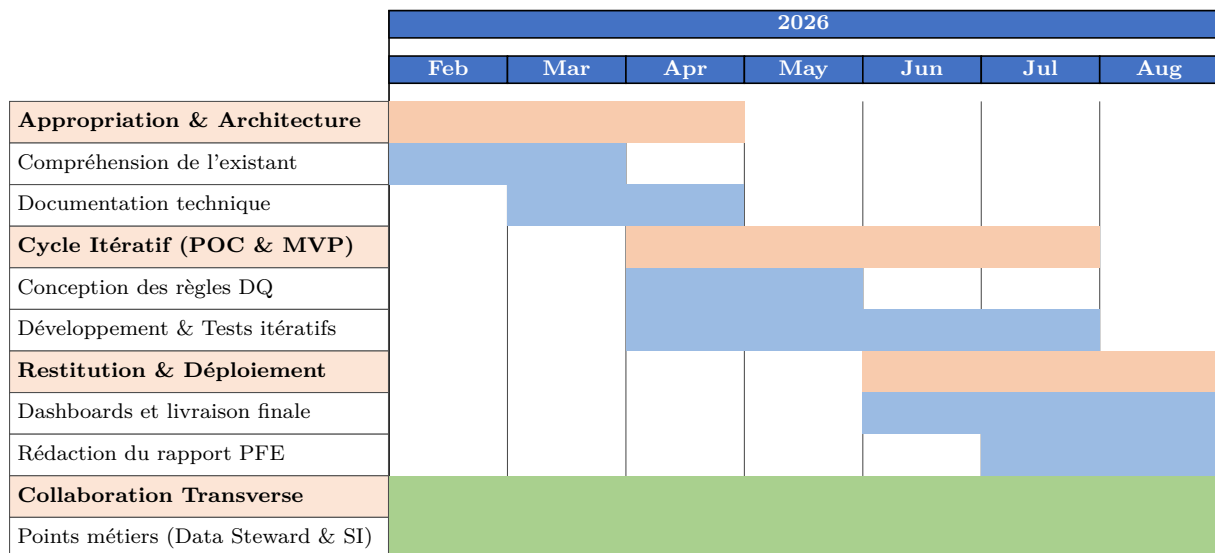


FIGURE 4 – Planning prévisionnel du projet

Conclusion du chapitre

Pour conclure, CIH BANK est une institution bancaire marocaine majeure, fortement structurée et engagée dans la valorisation de la donnée à travers son Data Office. L'étude de l'existant a permis de mettre en évidence les processus actuels de gestion des données et leurs limites, ce qui justifie la nécessité d'un pipeline automatisé de contrôle qualité. Ces éléments constituent la base de notre projet et nous permettent d'entamer sereinement la phase de spécification des besoins dans le chapitre suivant.

Chapitre 2

Analyse des Besoins et Spécification

Introduction du chapitre

Après avoir décrit le cadre général du projet et analysé le fonctionnement de la solution existante au sein de CIH BANK, ce chapitre se focalise sur la définition claire et rigoureuse des besoins. Nous y détaillons d’une part les besoins fonctionnels, c’est-à-dire les fonctionnalités attendues du pipeline de Data Quality, et d’autre part les besoins non fonctionnels, liés aux exigences techniques, de performance et de sécurité. Enfin, nous présentons la modélisation fonctionnelle à travers la spécification des cas d’utilisation et de leurs acteurs.

2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable de réaliser pour répondre aux problématiques de qualité des données identifiées sur le domaine Tiers. Ces problématiques, transverses à l’ensemble des tables du référentiel, seront illustrées par la table `np_description` et ses 40 colonnes (`name`, `firstname`, `civility`, `sex`, `birthdate`, `nationalitycountry`, `fatcaidentificationnumber`, etc.). Les besoins se répartissent en quatre fonctionnalités majeures :

2.1.1 Automatisation et planification des contrôles

- **Sous-besoin 1.1 : Déclenchement automatique** : Le système doit lancer l’évaluation de la qualité des données de manière planifiée, immédiatement après l’exécution complète du batch quotidien d’ingestion des tables sources Oracle NOVA (qui concernent environ 15 tables préfixées par `CCM_*` pour le périmètre Tiers) (via un nouveau DAG dédié s’exécutant à 00h00, séparé du DAG d’ingestion `dailyow22` de 22h00) ,
- **Sous-besoin 1.2 : Orchestration intégrée** : L’exécution du scan de qualité doit être orchestrée au sein d’Apache Airflow, permettant de suivre son statut d’exécution (Succès/Échec) et de consulter les logs en temps réel.

2.1.2 Configuration déclarative des règles de qualité

- **Sous-besoin 2.1 : Moteur de règles dynamique** : Le système doit permettre de définir les contrôles de qualité (Missing, Invalid, Freshness, Consistency, Uniqueness) dans des fichiers de règles déclaratifs séparés du code source ,

- **Sous-besoin 2.2 : Graduation des seuils et des alertes** : Pour chaque contrôle, le système doit pouvoir évaluer des seuils paramétrables (ex : taux de valeurs manquantes inférieur à 5%) et générer un statut de warning ou d’error selon la gravité de l’écart ,
- **Sous-besoin 2.3 : Gestion des métadonnées** : Chaque règle doit être documentée avec des attributs métier (nom de la règle, dimension DQ associée, domaine KYC ou CSP, criticité, etc.).

2.1.3 Historisation et traçabilité des indicateurs

- **Sous-besoin 3.1 : Persistance journalière** : À chaque exécution, le système doit sauvegarder les métriques calculées et les résultats de chaque contrôle de qualité dans des tables d’historique de scores (base Hive dédiée `dq_results`), indexée par date d’exécution ,
- **Sous-besoin 3.2 : Identification des anomalies (Remédiation)** : Le système doit capturer de manière détaillée la liste des enregistrements clients en anomalie en stockant leur identifiant technique (UUID) et leur radical (numéro client), ainsi que le libellé du contrôle en échec et sa criticité. Ces informations doivent être stockées dans une table d’anomalies pour actions correctives.

2.1.4 Restitution visuelle et aide à la décision

- **Sous-besoin 4.1 : Exposition des données** : Les résultats d’historisation de la qualité doivent être exposés de manière transparente sous forme de vues SQL pour les outils décisionnels ,
- **Sous-besoin 4.2 : Suivi de l’évolution temporelle** : Le tableau de bord doit permettre de suivre la tendance d’évolution des scores DQ dans le temps (30 derniers jours) ,
- **Sous-besoin 4.3 : Priorisation des remédiations** : Le tableau de bord doit fournir une liste opérationnelle des clients en anomalie triée par criticité décroissante, exportable par les Data Stewards pour traitement correctif dans le système source NOVA.

2.2 Besoins non fonctionnels

Les besoins non fonctionnels expriment les exigences qualitatives, opérationnelles et techniques que la solution doit respecter pour être viable et performante dans l’environnement de production de CIH BANK :

2.2.1 Performance

Le Référentiel Tiers de la banque regroupe **plusieurs millions** de dossiers clients (personnes physiques). Les règles de cohérence inter-colonnes ou de doublons nécessitent des jointures lourdes. Le moteur DQ doit être capable d’interroger et d’évaluer ce volume massif de données en un temps minimal (moins de 15 minutes) sans dégrader les ressources du Data Lake. Cela justifie l’utilisation d’un moteur de calcul distribué (Apache Spark) pour le traitement.

2.2.2 Disponibilité

L'exécution des contrôles de qualité ne doit pas bloquer la disponibilité des données pour les outils de reporting en cas d'anomalie. Si un seuil de qualité est franchi ou si le scan échoue, le pipeline d'exposition (Dremio/Tableau) doit continuer à fonctionner en mode dégradé en signalant l'alerte. De plus, la tâche DQ doit s'exécuter même si certaines tables d'ingestion n'ont été chargées que partiellement.

2.2.3 Maintenabilité

L'ajout de nouvelles règles ou l'intégration de nouvelles colonnes du Référentiel Tiers (par exemple pour de nouveaux critères réglementaires ou pour les personnes morales) doit se faire sans modification du code Python principal de la solution. Les fichiers de configuration YAML et la structure des tables Hive de résultats doivent être conçus de manière générique pour supporter l'extensibilité.

2.2.4 Sécurité

Les détails d'anomalies de qualité des données contiennent des informations clients sensibles (**name**, **firstname**, radicaux). Seuls les collaborateurs habilités du Data Office (Data Stewards, Data Project) et les Data Owners des départements concernés doivent être autorisés à accéder aux détails des anomalies. La solution s'appuie sur les mécanismes de sécurité existants de la plateforme :

- **Authentification Kerberos** : L'ensemble des composants du cluster Cloudera (HDFS, Hive, Spark) sont sécurisés par Kerberos, garantissant l'identité de chaque service et utilisateur ,
- **Autorisation Apache Ranger** : Les politiques d'accès aux tables Hive et aux espaces HDFS sont gérées de manière centralisée via Ranger, permettant un contrôle fin par rôle ,
- **Annuaire LDAP / Active Directory** : L'authentification des utilisateurs Dremio et Tableau est déléguée à l'annuaire d'entreprise, assurant une gestion unifiée des identités.

2.2.5 Conformité Réglementaire et Normative

Le système de gestion de la qualité des données doit s'inscrire dans un cadre strict de conformité légale et normative. Concernant la confidentialité et le traitement des données personnelles (noms, dates de naissance, identifiants), la solution doit impérativement respecter les exigences de la **loi 09-08** promulguée par la CNDP (Commission Nationale de contrôle de la Protection des Données à caractère personnel) au Maroc. De plus, la banque gérant potentiellement des clients résidant en Europe, la solution s'aligne sur les directives rigoureuses du **RGPD** (Règlement Général sur la Protection des Données). Enfin, la démarche globale de fiabilisation et de sécurisation de l'information s'appuie sur les standards internationaux, notamment la norme **ISO 8000** (standard mondial pour la qualité des données) et la norme **ISO/IEC 27001** (management de la sécurité de l'information).

2.3 Modélisation des cas d'utilisation

2.3.1 Acteurs du système

L'intégration de la solution de Data Quality au sein de CIH BANK repose sur une étroite collaboration entre les experts métier, l'équipe technique et les systèmes d'information. Le tableau 4 présente les cinq acteurs clés du système.

TABLE 4 – Acteurs du système de Data Quality

Acteur	Type	Rôle dans le système DQ
Équipe Data (DE / Data Steward)	Interne	Organise les ateliers d'alignement, formalise les règles dans les fichiers YAML Soda Core et implémente les flux techniques.
Métiers (Data Owners)	Humain	Experts fonctionnels qui définissent et valident les règles de qualité, consultent le dashboard et remédient aux anomalies.
Équipe SI	Appui	Valide la faisabilité technique des règles vis-à-vis du modèle physique Oracle NOVA.
Airflow (Orchestrateur)	Système	Déclenche automatiquement le scan DQ chaque soir après l'ingestion (nouveau DAG à 00h00).
Oracle NOVA (Système source)	Externe	Récepteur final des corrections de données opérées par les gestionnaires suite aux alertes DQ.

2.3.2 Diagramme global des cas d'utilisation

Le diagramme ci-dessous synthétise graphiquement les cas d'utilisation du système de Data Quality et les interactions avec les différents acteurs impliqués dans ce cycle de gouvernance.

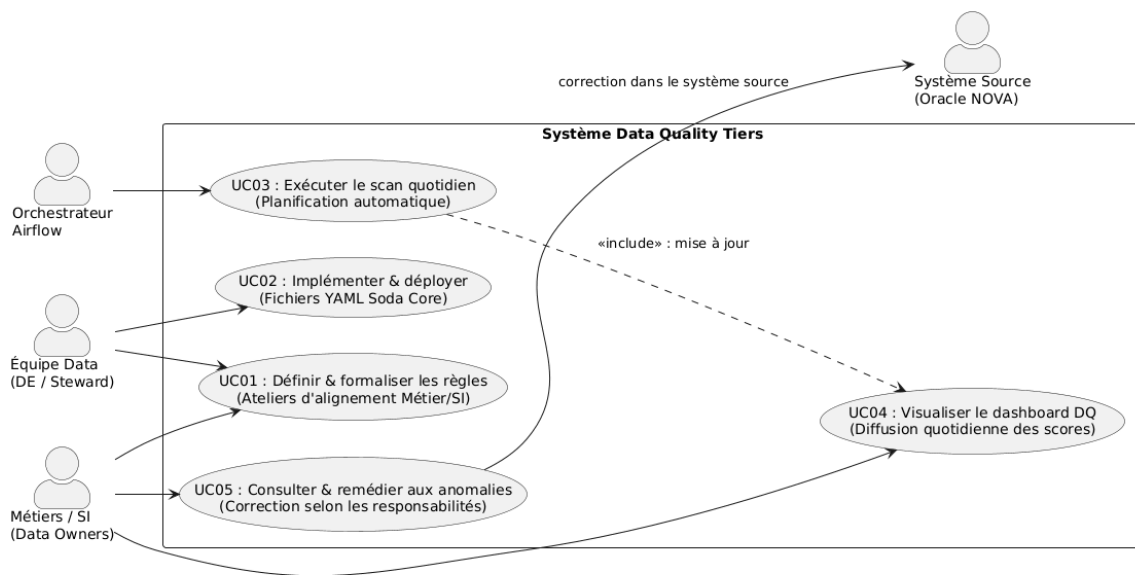


FIGURE 5 – Diagramme global des cas d'utilisation de la solution DQ Tiers

2.3.3 Description textuelle des cas d'utilisation principaux

Nous détaillons ci-dessous le déroulement des trois phases principales constituant le cycle de vie de la qualité des données de notre solution.

Cas d'utilisation 01 : Définir et formaliser les règles de qualité

TABLE 5 – Description du cas d'utilisation 01

Champ	Description
Titre	Définir et formaliser les règles de qualité
Acteurs	Équipe Data (Steward / DE), Métiers (Data Owners), Équipe SI
Préconditions	Cadrage fonctionnel du domaine Tiers défini
Scénario nominal	1. L'équipe Data organise des ateliers d'alignement avec les experts Métier et l'équipe SI. 2. Les parties s'accordent sur le catalogue des contrôles (complétude KYC, cohérence CSP) et les seuils de tolérance. 3. L'équipe Data consigne les spécifications dans un document de cadrage. 4. Le Data Engineer implémente les règles dans les fichiers YAML Soda Core.
Postconditions	Les règles de qualité sont formalisées et déployées dans le moteur Soda.

Cas d'utilisation 02 : Exécuter le scan de qualité (automatique)

TABLE 6 – Description du cas d'utilisation 02

Champ	Description
Titre	Exécuter le scan de qualité (automatique)
Acteur principal	Orchestrateur Airflow (Système)
Préconditions	Fin de l'ingestion quotidienne Sqoop des tables NOVA
Scénario nominal	1. Airflow lance automatiquement la tâche DQ de nuit. 2. Le moteur DQ Soda Core lit les fichiers YAML et exécute les requêtes sur Spark. 3. Les résultats agrégés et les détails des anomalies clients sont historisés dans la base Hive <code>dq_results</code>.
Postconditions	Les indicateurs et anomalies du jour sont prêts à être exposés.

Cas d'utilisation 03 : Consulter le dashboard et remédier aux anomalies

TABLE 7 – Description du cas d'utilisation 03

Champ	Description
Titre	Consulter le dashboard et remédier aux anomalies
Acteurs	Métiers (Parties prenantes), Data Stewards
Préconditions	Le scan DQ s'est exécuté de manière conforme
Scénario nominal	1. Le dashboard Tableau se rafraîchit automatiquement chaque matin. 2. Chaque partie prenante consulte le dashboard depuis son poste de travail. 3. Chaque acteur filtre et exporte la liste des anomalies de son périmètre. 4. Les acteurs procèdent à la correction des anomalies dans Oracle NOVA.
Postconditions	Les données corrigées dans NOVA sont ré-ingérées lors du prochain batch et le score DQ s'améliore automatiquement.

Conclusion du chapitre

Ce chapitre a permis de formaliser les besoins fonctionnels et non fonctionnels, définissant de manière précise les objectifs techniques et opérationnels du pipeline de qualité des données. La modélisation des acteurs et des cas d'utilisation reflète fidèlement le cycle de gouvernance de CIH BANK, où chaque partie prenante collabore d'abord à la définition des contrôles, puis intervient individuellement sur son périmètre pour corriger les anomalies identifiées au quotidien. Fort de ces spécifications, le chapitre suivant présente la conception globale et détaillée de la solution.

Chapitre 3

Conception de la Solution

Introduction du chapitre

Après avoir défini les besoins fonctionnels et non fonctionnels de notre système de contrôle de la qualité des données, ce chapitre présente la conception détaillée de la solution. Conformément aux exigences du Template 2, nous décrivons dans un premier temps la modélisation dynamique à travers des diagrammes de séquences, et d'activités. Dans un second temps, nous présentons la modélisation statique comprenant le diagramme de classes, le modèle relationnel, le dictionnaire de données et le catalogue complet des règles de qualité. Enfin, nous détaillons l'architecture logicielle (diagramme de composants) et matérielle (diagramme de déploiement) de l'application dans son environnement CIH BANK.

3.1 Modélisation dynamique

La modélisation dynamique permet de représenter les interactions temporelles entre les différents composants du système et l'évolution des états des données au cours de l'exécution du processus de contrôle.

3.1.1 Diagramme de séquences

Le diagramme de séquences de la figure 6 détaille le flux d'exécution séquentiel d'un scan de qualité de données. Il illustre comment la tâche DQ intégrée dans Airflow coordonne le traitement entre la zone HDFS RAW, le moteur Soda Core et la base Hive de résultats, jusqu'à l'exposition finale vers Tableau.

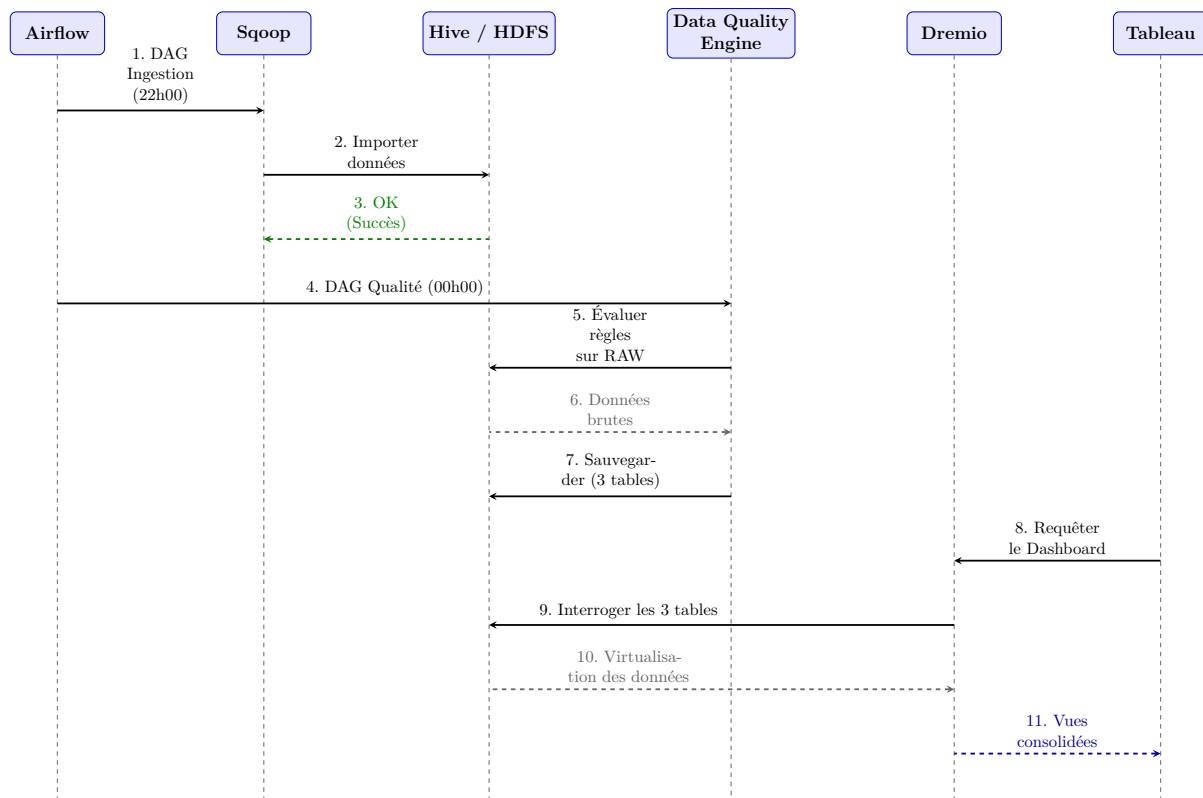


FIGURE 6 – Diagramme de séquences du pipeline de contrôle et restitution DQ

3.1.2 Diagramme d’activités

Le diagramme d’activités présenté dans la figure 7 décrit le processus de bout en bout de traitement de la qualité, depuis le chargement initial des données en zone RAW HDFS jusqu’à la remédiation effective dans l’application source Oracle NOVA.

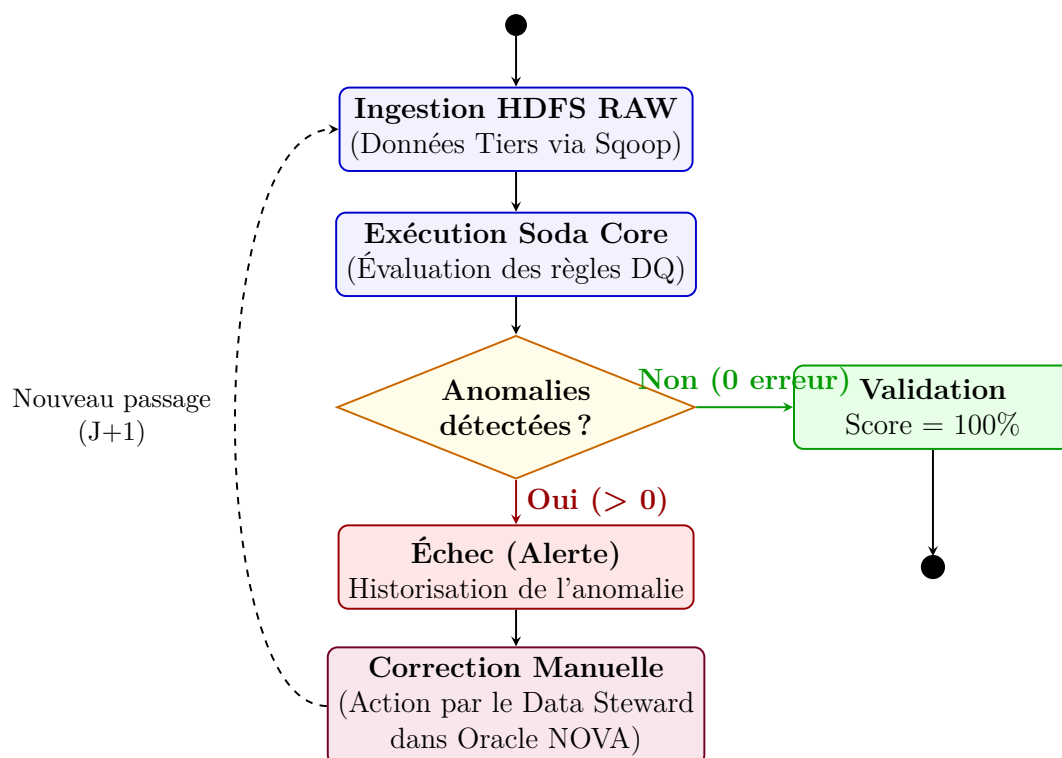


FIGURE 7 – Diagramme d’activités du workflow de remédiation DQ

3.2 Modélisation statique

La modélisation statique définit la structure fixe du système, ses composants de données, le schéma relationnel de stockage de l'historique de qualité, ainsi que le dictionnaire des métadonnées associées.

3.2.1 Modélisation de la table de faits

Étant donné que la solution s'articule autour de la manipulation de données (Data Management) et non d'un développement orienté objet, nous remplaçons le diagramme de classes traditionnel par une modélisation explicite de la table de faits.

La figure 8 illustre la structure des données générée par notre moteur de qualité. Le moteur Soda Core produit un ensemble de trois tables physiques liées par l'identifiant d'exécution (`run_id` et `query_id`). Ces trois tables sont ensuite unifiées au sein de Dremio pour exposer une grande table de faits consolidée (21 colonnes) servant de source unique de vérité pour le reporting BI.

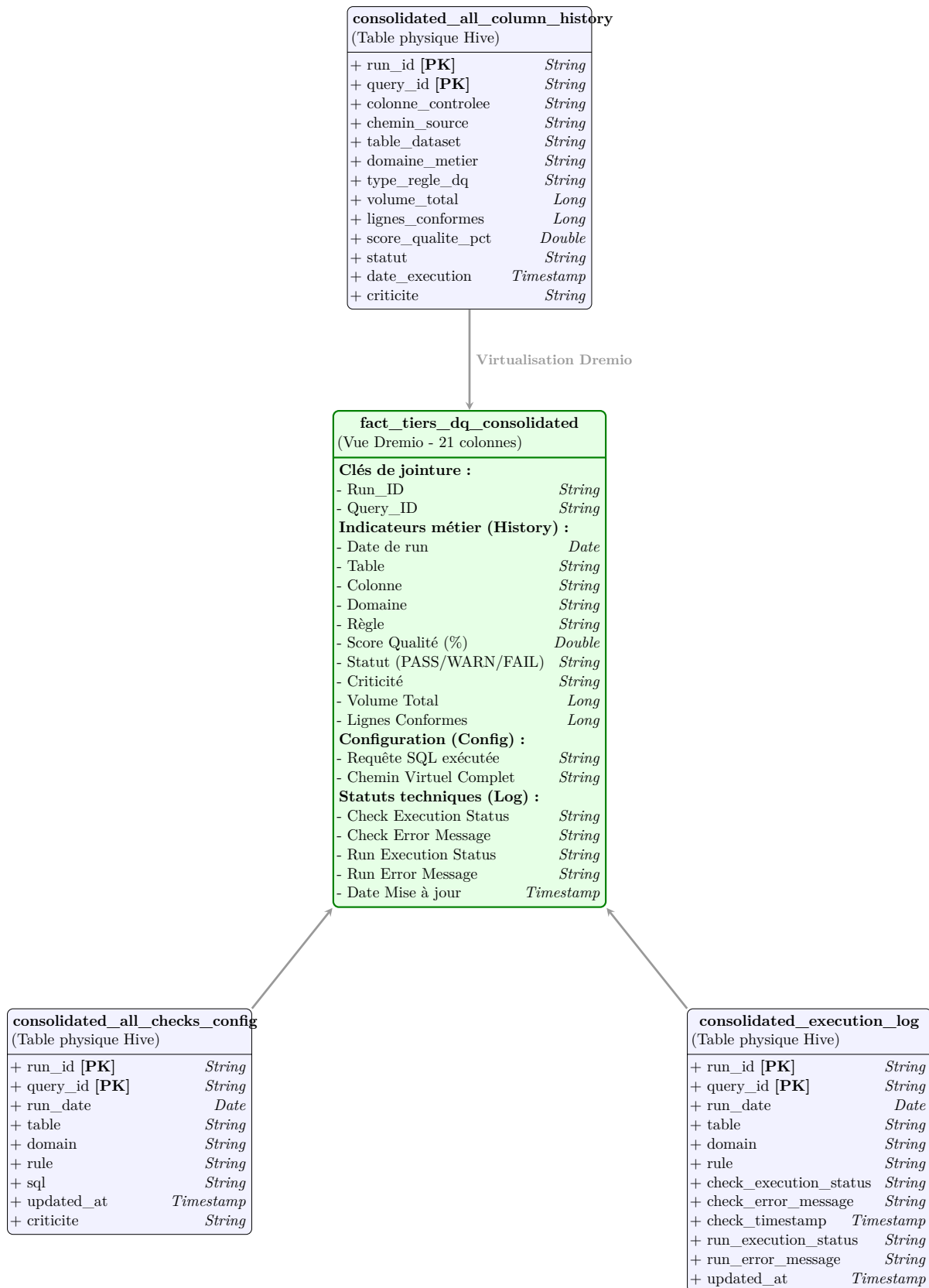


FIGURE 8 – Modélisation de la grande table de faits `fact_tiers_dq_consolidated` et ses 3 tables sources

3.2.2 Modèle relationnel

Pour persister les résultats d'évaluation de la qualité des données, nous concevons un schéma physique relationnel indépendant au sein de la base de données Hive, basé sur trois tables distinctes :

- **consolidated_all_column_history** : Table principale d'historisation des indicateurs et scores de qualité par exécution, table et colonne. Elle inclut la criticité métier du contrôle.
- **consolidated_all_checks_config** : Table de configuration stockant la requête SQL générée par le moteur pour chaque règle évaluée, ainsi que sa criticité.
- **consolidated_execution_log** : Table de journalisation technique traçant le succès ou l'échec d'exécution du contrôle.

3.2.3 Dictionnaire de données

Les attributs clés de ces trois tables sont consolidés via Dremio pour fournir un ensemble de 21 colonnes à Tableau. Voici les colonnes principales exploitées :

TABLE 8 – Dictionnaire simplifié des tables de faits de Qualité des Données

Colonne	Table source	Description / Rôle
run_id	Toutes	Identifiant unique de l'exécution du processus DQ.
query_id	Toutes	Identifiant de la requête ou de la règle évaluée.
type_regle_dq	column_history	Dimension DQ (ex : Completude, Conformite, Coherence...).
criticite	column_history / checks_config	Niveau de criticité du contrôle (CRITIQUE, HAUTE, MOYENNE).
domaine_metier	column_history	Domaine métier visé (PTP-TIE-POR).
score_qualite_pct	column_history	Taux de conformité en pourcentage.
statut	column_history	État d'évaluation métier de la règle (PASS, WARN, FAIL).
sql	checks_config	Requête SQL exécutée par le moteur SparkSQL.
check_execution_status	execution_log	Statut d'exécution technique du job (SUCCESS/ERROR).

3.2.4 Catalogue des règles de qualité

Les contrôles de qualité s'appliquent sur l'ensemble des données du domaine Tiers (informations signalétiques, données socioprofessionnelles, etc.). Afin d'offrir une structuration claire et directement exploitable, le catalogue des règles de gestion est organisé par **dimension de Data Quality** : Complétude, Conformité, Cohérence, Unicité et Actualité / Pipeline.

L'établissement de ce catalogue découle d'une analyse exploratoire approfondie menée sur un échantillon réel de 5 000 dossiers de personnes physiques issus des tables du périmètre (**environ 15 tables CCM_*, dont ccm_npdescription prise ici à titre d'exemple illustratif**). Cette analyse a révélé des anomalies typiques et récurrentes de saisie au sein du système opérant Oracle NOVA, ce qui a directement guidé le choix et le paramétrage de nos dimensions de contrôle :

- **Le phénomène des dates de naissance génériques** : Une forte concentration de dates de naissance au premier janvier (01/01/YYYY) a été identifiée (par exemple

1947-01-01, 1961-01-01), traduisant une saisie par défaut dans NOVA lorsque seule l'année de naissance est connue. Cela justifie notre règle de validité sur `birthdate` avec un seuil de tolérance de 15% ,

- **Le codage numérique des villes** : Le champ `birthcity` contient des codes numériques (comme 787) au lieu de libellés textuels, nécessitant un contrôle de validité par rapport au référentiel des villes pour assurer la cohérence et la résolubilité des codes ,
- **La cohérence conditionnelle FATCA** : L'identifiant FATCA (`fatcaidentificationnumber`) est majoritairement nul, ce qui est cohérent pour les citoyens marocains (`nationalitycountry` = MA), mais doit être obligatoirement renseigné pour les clients ayant la nationalité ou une double nationalité américaine.

Dimension Complétude

La dimension de complétude mesure le taux de renseignement des attributs obligatoires pour assurer l'exploitabilité opérationnelle des dossiers clients.

TABLE 9 – Dimension Complétude — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
<code>name</code>	0 valeur nulle tolérée	CRITIQUE	Audit dossiers - Blocage saisie agence
<code>firstname</code>	0 valeur nulle tolérée	CRITIQUE	Audit dossiers - Blocage saisie agence
<code>birthdate</code>	0 valeur nulle tolérée	CRITIQUE	Saisie obligatoire avec justificatif officiel
<code>nationality_country</code>	0 valeur nulle tolérée	HAUTE	Renseigner le code pays de nationalité
<code>professional_sector</code>	Taux de valeurs nulles < 5%	HAUTE	Renseigner le secteur d'activité de l'employeur
<code>occupation</code>	Taux de valeurs nulles < 5%	HAUTE	Préciser la profession exacte du client
<code>socioprofessional_category</code>	Taux de valeurs nulles < 5%	HAUTE	Renseigner la catégorie socioprofessionnelle (CSP)
<code>professional_situation</code>	Taux de valeurs nulles < 5%	HAUTE	Renseigner le statut de la situation professionnelle
<code>employer</code>	Taux de valeurs nulles < 10%	HAUTE	Renseigner le nom de l'entreprise employeuse
<code>politically_exposed</code>	0 valeur nulle tolérée (PEP obligatoire)	CRITIQUE	Renseigner la déclaration d'exposition politique

Dimension Conformité

La dimension de validité évalue le respect des formats de saisie, des plages de valeurs autorisées et la conformité par rapport aux référentiels métier (BAM, ISO).

TABLE 10 – Dimension Conformité — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
name, firstname	Longueur ≥ 2 , aucun chiffre ou caractère spécial	HAUTE	Nettoyage et normalisation dans NOVA
birthdate	Taux de dates génériques au 01/01 $< 15\%$	HAUTE	Recherche de la date réelle sur pièce d'identité
civility	Valeurs restreintes à (M., MME, MLLE)	HAUTE	Restauration de la valeur selon pièces fournies
sex	Valeurs restreintes à (M, F)	HAUTE	Restauration de la valeur selon pièces fournies
birth_city	Doit exister dans le référentiel des villes BAM	MOYENNE	Mapping et correction via la table de transcodage
professional_sector	Code numérique valide dans le référentiel sectoriel	MOYENNE	Transcodage vers les codes officiels BAM
professional_situation	Valeurs restreintes à (SAL, FON, TRI, AUT, EMP, NDE)	HAUTE	Remplacer par un code référentiel valide
employer	Détection des valeurs parasites ('AUTRE', 'LUI ME-ME')	HAUTE	Normaliser : mapper 'LUI MEME' vers auto-entrepreneur

Dimension Cohérence

La dimension de cohérence vérifie l'absence de contradiction logique entre plusieurs attributs interdépendants.

TABLE 11 – Dimension Cohérence — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
sex + civility	Genre M $\neq$ MME/MLLE ; Genre F $\neq$ M.	CRITIQUE	Résolution manuelle des incohérences de saisie
fatca_identification_number	Si nationalité ou double nationalité = US, 0 valeur nulle	CRITIQUE	Déclaration réglementaire FATCA immédiate
employer + professional_situation	Si situation = NDE (sans emploi) ou TRI (retraité), employeur doit être NULL	HAUTE	Annuler le nom d'employeur incohérent dans NOVA

Dimension Unicité

La dimension d'unicité contrôle l'absence de doublons d'enregistrements et garantit l'unicité des identifiants principaux du référentiel.

TABLE 12 – Dimension Unicité — Données Tiers (Référentiel Tiers)

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
oid / radical	Unicité stricte du numéro de radical client (0 doublon)	CRITIQUE	Fusion des doublons clients dans le référentiel
ordnum	Un seul enregistrement actif principal (main=1) par tiers	HAUTE	Éliminer les contrats de travail multiples actifs

Dimension Actualité et Pipeline

La dimension de fraîcheur s'assure du bon cadencement de l'ingestion quotidienne et de l'intégrité de la chaîne technique.

TABLE 13 – Dimension Actualité et Pipeline — Flux Batch

Colonne(s)	Description Règle / Seuil	Criticité	Action corrective
batch_datetime	Retard d'ingestion HDFS RAW < 26h	HAUTE	Alerte infrastructure Big Data et relance Sqoop
row_count	Écart de volumétrie entre Oracle NOVA et Hive RAW < 0.1%	CRITIQUE	Rejouer le DAG d'ingestion Sqoop
code_segment	Cohérence entre segment client et code_marche	HAUTE	Recalculer le segment par défaut

3.3 Architecture de l'application

L'architecture présente la structure logique de la solution DQ et son déploiement physique sur l'infrastructure de production de CIH BANK.

3.3.1 Architecture logicielle (diagramme de composants)

L'architecture logicielle de notre solution s'articule autour de modules découplés. Le diagramme de la figure 9 illustre les interactions entre les modules d'ingestion, de stockage, de contrôle, de virtualisation et de restitution de la solution.

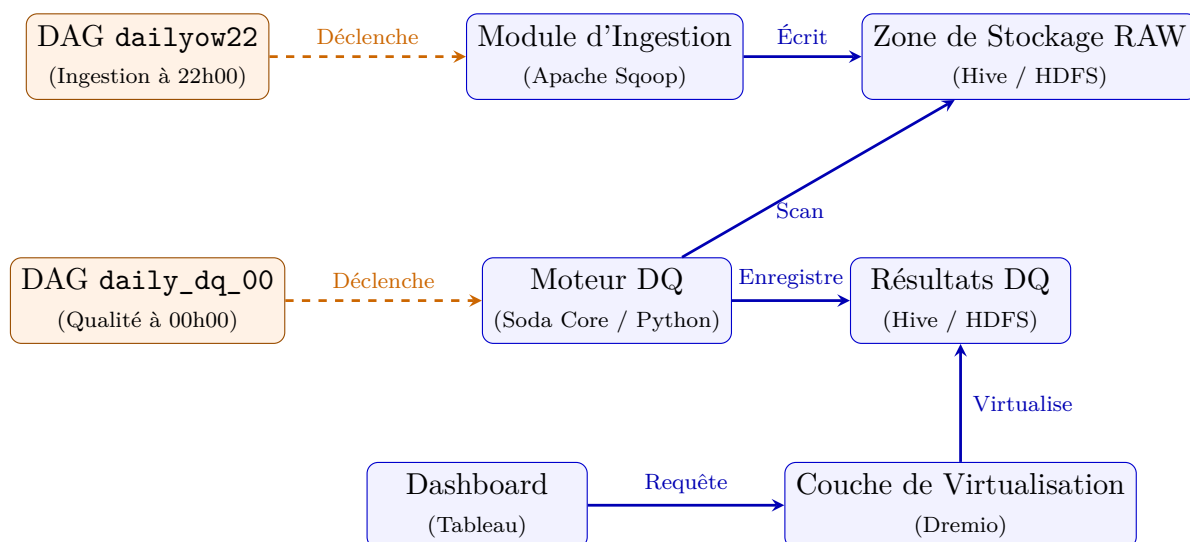


FIGURE 9 – Diagramme de composants logiciels de la solution DQ

Les composants clés de cette architecture logicielle sont détaillés ci-dessous :

Conception du moteur DQ — Soda Core

Le moteur de contrôle s'appuie sur le framework open source Soda Core, spécifiquement la bibliothèque Python `soda-core` et son connecteur Spark `soda-core-spark`. Afin d'assurer une exécution optimale dans notre écosystème Big Data, la conception logicielle repose sur les principes d'intégration suivants :

- **Encapsulation PySpark et Vues Temporaires** : Le script d'exécution principal (`dq_tiers_checks.py`) initialise une session Spark locale. Pour chaque table de la zone RAW à contrôler (parmi la quinzaine de tables du périmètre CCM_*, telles que `ccm_npdescription` et `ccm_job` citées en exemple), la session Spark charge la table sous forme de DataFrame PySpark, puis l'enregistre en tant que vue temporaire en mémoire via la méthode `createOrReplaceTempView()`. Cela permet à Soda Core d'exécuter l'analyse de qualité directement sur ces vues mémoire sans poser de verrous ou d'accès concurrents sur les tables Hive physiques ,
- **Définition déclarative des règles (SodaCL)** : Les contrôles de qualité sont externalisés dans des fichiers de règles YAML exploitant le langage déclaratif SodaCL. Les règles fonctionnelles sont regroupées au sein d'un unique fichier métier `checks_tiers.yml`, tandis que les vérifications techniques sont isolées dans `checks_pipeline.yml` ,
- **Extraction programmatique et typage des métadonnées** : Après l'exécution du scan, le script Python exploite les structures d'API de Soda Core (l'objet `Scan`) pour parcourir la liste des vérifications évaluées. Pour chaque règle, le script extrait dynamiquement le résultat (succès/échec), la valeur de la métrique, le seuil, la dimension (Completeness, Validity, Consistency), le domaine et la criticité (définis en tant qu'attributs personnalisés dans le YAML). Un score de conformité sur 100 est attribué pour chaque règle ,
- **Persistance partitionnée dans trois tables Hive** : Après l'exécution du scan, le script consolide les résultats et écrit dans trois tables physiques Hive distinctes : `consolidated_all_column_history` pour les résultats, `consolidated_all_checks_config` pour la configuration et les requêtes SQL générées, et `consolidated_execution_log` pour la journalisation technique.

Conception de l'orchestration Airflow

La planification, le déclenchement et le monitoring des scans de qualité sont gérés par un **nouveau DAG Airflow indépendant**, spécifiquement dédié au contrôle DQ. Afin de découpler totalement l'ingestion de la vérification de la qualité, ce nouveau DAG est planifié pour s'exécuter quotidiennement à **00h00**. Cela laisse une marge de sécurité de deux heures après le déclenchement du DAG d'ingestion Sqoop principal (**dailyow22**, prévu à 22h00). L'architecture d'exécution repose sur un déclenchement direct du moteur d'évaluation Python Soda Core depuis Airflow. Cette séparation modulaire assure qu'une défaillance dans le DAG d'ingestion ne bloque pas ou ne corrompt pas la supervision de la qualité.

Conception du dashboard Tableau

La restitution visuelle est structurée dans un dashboard Tableau interactif connecté en direct au dataset virtuel unique de Dremio. L'intelligence analytique (calcul des scores quotidiens moyens, taux de conformité globaux, regroupements par dimension de qualité ou par date) est configurée à l'aide de feuilles de calcul (sheets) et de filtres dynamiques directement dans Tableau Desktop, tirant parti de la table de faits unique. Le dashboard est structuré en quatre onglets fonctionnels :

1. **Vue Exécutive** : Présente les KPIs de conformité globaux et les alertes critiques pour les Data Owners et la Direction ,
2. **Détail Métier** : Permet une analyse fine par colonne et par dimension de qualité sur le domaine métier Tiers ,
3. **Détail CSP** : Restitue les scores de conformité socioprofessionnelle et l'évaluation des valeurs parasites ,
4. **Anomalies & Remédiation** : Présente une synthèse des règles et colonnes présentant des taux d'anomalies élevés (filtrée sur **state != 'pass'**), aidant les Data Stewards à cibler les actions correctives collectives dans NOVA.

Pour des raisons évidentes de sécurité et de conformité réglementaire (notamment les directives de Bank Al-Maghrib et la protection des données personnelles), toutes les données nominatives et les radicaux clients affichés dans les onglets opérationnels font l'objet d'un masquage ou d'un floutage systématique, garantissant la stricte confidentialité des informations sensibles.

3.3.2 Architecture matérielle (diagramme de déploiement)

Le diagramme de déploiement de la figure 10 modélise l'architecture système et l'infrastructure distribuée de production sur laquelle est déployée notre solution au sein de CIH BANK.

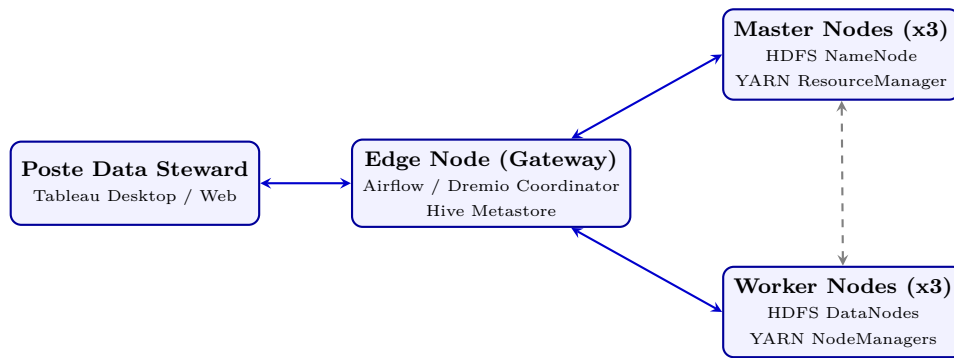


FIGURE 10 – Architecture matérielle et déploiement dans le cluster Big Data

Le cluster Big Data Cloudera CDP 7.1.7 de production de CIH BANK est structuré autour de 8 nœuds distribués, configurés de la manière suivante :

- **1 Nœud Edge (Gateway) principal** : Ce serveur sert de point d'entrée pour l'administration et l'orchestration des jobs. Il héberge les workers d'Apache Airflow, le coordinateur Dremio, ainsi que le service Hive Metastore. C'est sur ce nœud qu'est installé l'environnement virtuel Python contenant les packages `soda-core` et `soda-core-spark`,
- **3 Nœuds Masters** : Ils assurent la haute disponibilité du stockage (HDFS NameNodes configurés en Actif/Standby et JournalNodes) et la coordination de l'ordonnancement distribué. Ils hébergent également le YARN ResourceManager pour la répartition des ressources du cluster,
- **3 Nœuds Workers** : Ces serveurs gèrent le stockage local physique sous HDFS (DataNodes) et disposent des YARN NodeManagers pour allouer des conteneurs de calcul lors de l'exécution distribuée de nos requêtes PySpark et Dremio Executors.

Orchestration technique et protocole de sécurité Kerberos

Afin de sécuriser les accès aux données confidentielles du domaine Tiers de CIH BANK, l'exécution de la solution de qualité s'intègre au protocole de sécurité global du cluster :

- **Authentification Kerberos** : Le cluster Cloudera étant sécurisé par Kerberos, l'autorisation d'accès aux répertoires HDFS RAW nécessite un ticket d'authentification valide. Avant de lancer le traitement, la tâche Airflow utilise le fichier keytab chiffré (`tiers_dq.keytab`) associé au compte de service du domaine Tiers (`tiers_dq_service`) pour générer et renouveler automatiquement le ticket Kerberos via la commande système `kinit`,
- **Soumission du job Spark sur YARN** : Le script Python principal est exécuté en mode `spark-submit` avec le gestionnaire de ressources YARN configuré en mode `client` depuis le nœud Edge. Le driver Spark s'exécute ainsi localement sur le Gateway Node (dans le process Airflow), tandis que YARN ResourceManager alloue de manière distribuée des exécuteurs Spark au sein des nœuds Workers. Ces exécuteurs lisent et traitent en parallèle les partitions des fichiers HDFS de la zone RAW Hive.

3.3.3 Technologies de l'écosystème utilisées

Pour s'insérer au mieux au sein du système d'information décisionnel et Big Data de CIH BANK, notre solution exploite les briques technologiques de l'écosystème du Data Lake. Ces technologies sont résumées ci-dessous.



Oracle NOVA — Système opérant source

Base de données relationnelle hébergeant le Core Banking de la banque. Il s'agit du point d'entrée unique des données des clients physiques et moraux (Référentiel Tiers) et de la cible finale de correction.



MobaXterm — Client SSH et terminal distant

Outil d'administration système tout-en-un fournissant un terminal avancé pour accéder de manière sécurisée aux serveurs Linux de la banque. Il s'avère indispensable pour configurer l'environnement, déployer les scripts et tester les pipelines en ligne de commande directement sur le nœud Edge du cluster.



Apache Sqoop — Ingestion batch Oracle vers HDFS

Technologie d'ingestion permettant le transfert en masse des tables NOVA vers l'infrastructure Hadoop de manière planifiée.



HDFS / Apache Hive — Plateforme de stockage et de requêtage

HDFS assure le stockage persistant distribué tandis que Hive structure ces répertoires sous forme de tables SQL relationnelles cibles pour nos contrôles.



Apache Spark (Spark SQL) — Moteur de traitement distribué

Framework de calcul distribué in-memory utilisé par Soda Core pour exécuter les requêtes de contrôle de qualité massives sur les données Hive avec des performances optimales.



Soda Core — Moteur de contrôle de la qualité (Python)

Framework de validation de données déclaratif basé sur des fichiers YAML de règles de gestion et exécuté programmatiquement sur les tables Hive RAW.



Apache Airflow — Orchestrateur de tâches

Outil d'orchestration permettant de planifier, ordonnancer et superviser l'exécution du flux d'ingestion et des scans de qualité de manière centralisée.



Dremio — Moteur de virtualisation et d'exposition

Permet de virtualiser les tables de résultats Hive et d'exposer des vues SQL rapides et sécurisées, éliminant le besoin de dupliquer ou de transférer physiquement les volumes de données.



Tableau — Outil d'analyse décisionnelle et Visualisation

Solution BI standard de CIH BANK, connectée en direct à Dremio pour fournir un reporting réactif sur la qualité des tiers à l'intention des Data Owners.



GitLab — Gestion de versions, collaboration et suivi de projet

Plateforme DevOps centrale utilisée pour la gestion du code source (versioning Git), la revue de code (merge requests), et le suivi du backlog Data Engineering avec le Delivery Manager. GitLab centralise l'état d'avancement des tâches, assure la traçabilité des livraisons et garantit une gestion rigoureuse du cycle de vie des scripts DQ.

Conclusion du chapitre

Ce chapitre a décrit la conception complète de la solution de contrôle de la qualité du domaine Tiers. Nous avons structuré la dynamique d'exécution de nos scans et le cycle de vie des anomalies identifiées. La modélisation statique a fourni le diagramme de classes de nos indicateurs, le schéma relationnel et le catalogue complet des règles métier de la solution. Enfin, l'architecture logicielle (composants) et matérielle (déploiement) a été détaillée au sein de l'environnement de production. Le chapitre suivant présente l'implémentation physique et les résultats opérationnels obtenus.

Chapitre 4

Réalisation de la Solution

Introduction

Ce chapitre présente la réalisation concrète du pipeline de Data Quality sur le Référentiel Tiers de CIH Bank. Tous les contrôles Soda Core s'effectuent exclusivement sur la **zone RAW** du Data Lake (base Hive `raw_nova_referentieltiers`), qui constitue la copie exacte et non transformée des tables `CCM_*` du système opérant Oracle NOVA (le périmètre complet englobe environ 15 tables Tiers ; dans ce chapitre, les tables `ccm_npdescription` et `ccm_job` serviront d'exemples illustratifs).

Nous détaillerons successivement l'environnement technique de développement et l'accès via MobaXterm, l'implémentation complète du script Soda Core avec Spark, la création des trois tables Hive physiques de persistance, l'intégration dans un DAG Airflow indépendant, la mise en place de la vue virtualisée sous Dremio, et enfin la réalisation du dashboard Tableau.

4.1 Environnement technique de développement

4.1.1 Environnement Hadoop et accès distant

Le développement et les tests ont été réalisés dans l'environnement Hadoop sécurisé de CIH Bank. L'accès aux serveurs du cluster (nœud Edge) et l'exécution des commandes en ligne de commande (CLI) s'effectuent via l'outil **MobaXterm**. MobaXterm est un terminal avancé qui fournit toutes les fonctionnalités réseaux nécessaires (SSH, SFTP) pour se connecter au cluster Big Data.

Une fois connecté au nœud Edge, l'authentification sécurisée est garantie par le protocole **Kerberos** (commande `kinit`). L'accès aux tables Hive et aux fichiers HDFS est effectué en utilisant les comptes de service existants du domaine Tiers.

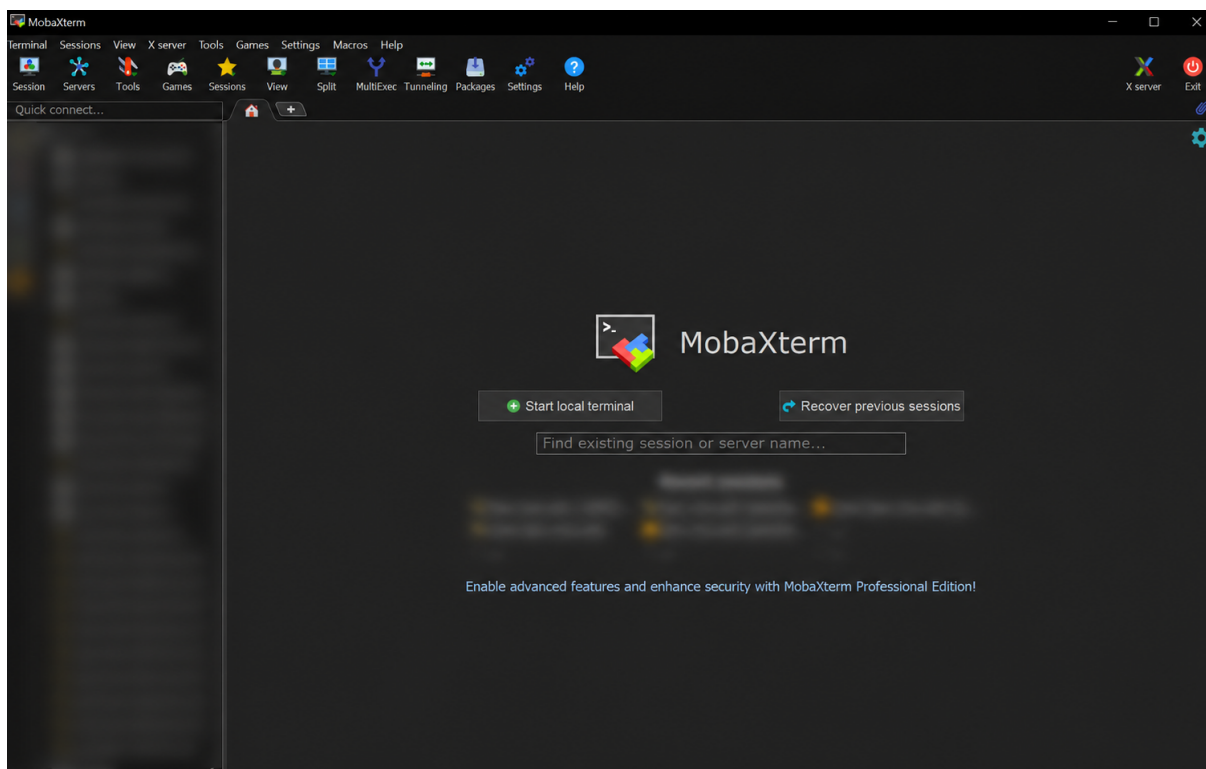


FIGURE 11 – Interface MobaXterm utilisée pour l'accès SSH au nœud Edge du cluster Cloudera

Les versions exactes des composants utilisés dans l'environnement de production sont les suivantes :

Composant	Version
Cloudera Data Platform (CDP)	Private Cloud Base 7.1.7
Apache Hadoop (HDFS / YARN)	3.1.1
Apache Hive	3.1.3
Apache Spark	3.2.0
Apache Airflow	2.5.1
Dremio	Enterprise Edition 2026
Python	3.9.13
Soda Core	soda-core-spark 3.0.45
Tableau Desktop	Professional Edition 2024.1.2

TABLE 14 – Versions des composants de l'environnement technique

4.1.2 Installation hors-ligne de Soda Core

La politique stricte de sécurité des systèmes d'information (PSSI) de CIH BANK impose une isolation réseau (air-gap) des serveurs du cluster Big Data, interdisant par conséquent tout accès direct à Internet. L'exécution standard de la commande `pip install` pour rapatrier les dépendances depuis le registre public PyPI est donc impossible.

L'installation de Soda Core et de son connecteur Spark a été réalisée via une procédure d'installation hors-ligne (Offline Wheel Installation) :

1. **Téléchargement des archives (Wheels) :** Sur un poste de travail disposant d'un accès à Internet, les archives des paquets `soda-core` et `soda-core-spark` ainsi que

l'ensemble de leurs sous-dépendances ont été téléchargées au format `.whl` dans un dossier dédié via la commande `pip download`.

2. **Transfert sécurisé via MobaXterm** : Le dossier d'archives a été transféré de manière sécurisée en SFTP vers le nœud Edge du cluster grâce à l'interface graphique de MobaXterm.
3. **Installation locale** : Sur le nœud Edge, les paquets ont été déployés dans l'environnement virtuel Python (venv) alloué à l'orchestrateur Airflow à l'aide de la commande `pip`, en forçant la recherche locale :

```
1 source /opt/airflow/venv_dq/bin/activate
2 pip install --no-index --find-links=./soda_wheels soda-core-spark
```

4.2 Implémentation du moteur DQ — Soda Core et Spark

4.2.1 Arborescence du projet DQ

Le moteur de Data Quality est structuré en un projet Python autonome, déployé sur le nœud Edge du cluster. L'arborescence suivante présente une vue simplifiée des fichiers essentiels :

```
1 dq_tiers/                                (Projet DQ - Nœud Edge)
2 |-- dq_tiers_checks.py  # Script principal : orchestration &
   persistence
3 |-- checks/
4 |   +-- checks_tiers.yml  # Règles DQ (KYC + CSP) en SodaCL
5 +-- utils/
6     +-- hive_writer.py    # Module de persistance des 3 tables
                           Hive
```

4.2.2 Principe de fonctionnement de Soda Core avec Spark

Contrairement à d'autres connecteurs SQL classiques (JDBC/ODBC) où Soda Core interroge directement la base de données, la bibliothèque `soda-core-spark-df` repose sur le traitement en mémoire. Le principe de fonctionnement adopté est le suivant :

1. Le script Python crée une **SparkSession** avec support Hive activé.
2. Il charge les tables Hive brutes de la zone RAW sous forme de **DataFrames PySpark**.
3. Ces DataFrames sont enregistrés en tant que **vues temporaires Spark** via `createOrReplaceTempView`.
4. Soda Core lit le fichier de règles YAML, le traduit automatiquement en **requêtes Spark SQL**, et exécute ces requêtes distribuées sur le cluster via YARN.
5. Les résultats sont extraits sous forme de dictionnaire Python, puis structurés et persistés dans **3 tables Hive physiques** en mode `append`.

4.2.3 Fichier de configuration YAML (checks_tiers.yml — exemple)

Les règles métier sont définies de manière déclarative en utilisant le langage **SodaCL** (Soda Check Language). Ce langage permet de définir des contrôles simples (complétude, validité) ou avancés (fraîcheur, unicité, règles SQL personnalisées). Toutes les règles du domaine Tiers (KYC et CSP) sont regroupées dans un fichier unique, présenté ici en extrait illustratif sur les deux tables d'exemple `ccm_npdescription` et `ccm_job` :

```
1 # checks_tiers.yml
2 # Regles Data Quality - Domaine Tiers (KYC & CSP)
3
4 checks for ccm_npdescription
5   # --- Complétude ---
6   - missing_count(name) = 0
7     name "KYC-01 | Complétude | Nom"
8     attributes
9       criticite CRITIQUE
10      dimension completeness
11      domaine Tiers
12
13   - missing_count(first_name) = 0
14     name "KYC-02 | Complétude | Prenom"
15     attributes
16       criticite CRITIQUE
17       dimension completeness
18
19   # --- Valider ---
20   - invalid_count(gender) = 0
21     valid values ["M", "F"]
22     name "KYC-06 | Valider | Genre hors referentiel"
23     attributes
24       criticite HAUTE
25       dimension validity
26       domaine Tiers
27
28   # --- Unicité ---
29   - duplicate_count(radical) = 0
30     name "KYC-07 | Unicité | Doublet radical client"
31     attributes
32       criticite CRITIQUE
33       dimension uniqueness
34
35 checks for ccm_job
36   # --- Complétude CSP ---
37   - missing_count(professional_sector) = 0
38     name "CSP-01 | Complétude | Secteur professionnel"
39     attributes
40       criticite CRITIQUE
41       dimension completeness
42
43   # --- Fraîcheur ---
44   - freshness(batch_datetime) < 26h
```

```

45     name "CSP-04 | Fraicheur | Retard batch superieur a 26h"
46     attributes
47         criticite HAUTE
48         dimension timeliness
49
50 # --- Regle SQL personnalisée (failed rows) ---
51 - failed rows
52     name "CSP-05 | Coherence | Employeur invalide pour ce
53     secteur"
54     fail query |
55         SELECT radical, employer, professional_sector
56         FROM ccm_job
57         WHERE employer = 'SANS EMPLOI' AND professional_sector !=
58         'CHOMAGE'
59     attributes
60         criticite MOYENNE
61         dimension consistency
62         domaine Tiers

```

Les fonctions SodaCL utilisées sont les suivantes :

- **missing_count** : Compte le nombre d'enregistrements où la colonne est nulle ou vide.
- **invalid_count** : Couplé à **valid values** ou **valid regex**, valide l'appartenance à un référentiel ou le respect d'un format.
- **duplicate_count** : Détecte les doublons sur une clé métier (ex : **radical**).
- **freshness** : Mesure le délai entre la valeur maximale d'une colonne date et l'instant d'exécution du scan.
- **failed rows** : Permet d'injecter du SQL pur pour des règles multi-colonnes complexes (cohérence, croisement de référentiels).

4.2.4 Traduction des règles SodaCL en Spark SQL

En interne, Soda Core identifie la cible (Spark) et traduit chaque règle SodaCL en une requête Spark SQL exécutée sur les vues temporaires. Voici trois exemples de cette traduction :

Exemple 1 — Complétude (missing_count**) :**

```

1 # >> Regle SodaCL :
2 - missing_count(name) = 0

```

⇓ Traduction automatique par Soda Core

```

1 -- >> Requete Spark SQL generee :
2 SELECT COUNT(*) AS missing_count
3 FROM ccm_npdescription
4 WHERE name IS NULL OR TRIM(name) = ''

```

Exemple 2 — Validité (invalid_count** avec référentiel) :**

```

1 # >> Regle SodaCL :

```

```

2 - invalid_count(gender) = 0
3   valid values ["M", "F"]

```

⇓ Traduction automatique par Soda Core

```

1 -- >> Requete Spark SQL genereee :
2 SELECT COUNT(*) AS invalid_count
3 FROM ccm_npdescription
4 WHERE gender NOT IN ('M', 'F') OR gender IS NULL

```

Exemple 3 — Fraîcheur (freshness) :

```

1 # >> Regle SodaCL :
2 - freshness(batch_datetime) < 26h

```

⇓ Traduction automatique par Soda Core

```

1 -- >> Requete Spark SQL genereee (etape 1) :
2 SELECT MAX(batch_datetime) AS last_batch FROM ccm_job
3 -- Soda calcule ensuite le delta : CURRENT_TIMESTAMP - last_batch
4 -- et compare le resultat en heures a la limite de 26h.

```

4.2.5 Script Python principal (dq_tiers_checks.py)

Le script Python principal orchestre l'exécution complète. Soda Core exécutant les checks sur Spark ne persiste pas automatiquement les résultats dans Hive (contrairement à l'intégration cloud). C'est le script Python qui extrait la matrice de résultats JSON via `get_scan_results()`, l'aplatit, puis structure les données en **3 DataFrames** dont les schémas respectent exactement les tables physiques définies en conception. Les écritures se font systématiquement en mode **append** pour historiser chaque run journalier.

```

1 # dq_tiers_checks.py
2 import datetime, uuid, argparse
3 from datetime import date
4 from pyspark.sql import SparkSession, Row
5 from soda.scan import Scan
6
7 def run_dq_checks(run_date: str):
8     # --- 1. SparkSession avec support Hive ---
9     spark = SparkSession.builder \
10         .appName("DQ_Tiers_Checks") \
11         .enableHiveSupport() \
12         .getOrCreate()
13
14     # --- 2. Chargement des tables RAW en vues temporaires Spark
15     ---
16     for table in ["ccm_npdescription", "ccm_job"]:
17         spark.sql(f"SELECT * FROM raw_nova_referentieltiers.{
18             table}") \

```

```

17         .createOrReplaceTempView(table)
18
19     # --- 3. Scan Soda Core ---
20     scan = Scan()
21     scan.set_data_source_name("spark_df")
22     scan.add_spark_session(spark)
23     scan.add_sodacl_yaml_file("checks/checks_tiers.yml")
24     scan.execute()
25     results = scan.get_scan_results()
26
27     run_id = str(uuid.uuid4())
28     now = datetime.datetime.now()
29     history_rows, config_rows, log_rows = [], [], []
30
31     for check in results.get("checks", []):
32         qid = check.get("identity", str(uuid.uuid4()))
33         outcome = check.get("outcome", "error")
34         attrs = check.get("attributes", {})
35         dataset = check.get("dataset", "")
36         col = check.get("column", "TRANSVERSE")
37         score = 100.0 if outcome=="pass" else (50.0 if outcome
38 == "warn" else 0.0)
39         total = check.get("metrics", {}).get("row_count", 0)
40         or 0
41         anomalies= int(check.get("metrics", {}).get("value", 0)
42         or 0)
43
44     # Recuperation SQL genere par Soda Core
45     sql_gen = next(
46         (q.get("sql", "") for q in results.get("queries", [])
47         if qid in q.get("name", "")), ""
48     )
49     exec_ok = (outcome != "error")
50
51     # -- Table 1 : consolidated_all_column_history --
52     history_rows.append(Row(
53         run_id = run_id,
54         query_id = qid,
55         colonne_controlee = col,
56         chemin_source = f"raw_nova_referentieltiers.{
57 dataset}",
58         table_dataset = dataset,
59         domaine_metier = attrs.get("domaine", "Tiers"),
60         type_regle_dq = attrs.get("dimension", "
61 completeness"),
62         volume_total = total,
63         lignes_conformes = total - anomalies,
64         score_qualite_pct = score,
65         statut = outcome.upper(),
66         date_execution = now,
67         criticite = attrs.get("criticite", "HAUTE")
68     ))
69

```

```

65     # -- Table 2 : consolidated_all_checks_config --
66     config_rows.append(Row(
67         run_id      = run_id,
68         query_id    = qid,
69         run_date    = run_date,
70         table       = dataset,
71         domain      = attrs.get("domaine", "Tiers"),
72         rule        = check.get("name", ""),
73         sql         = sql_gen,
74         updated_at  = now,
75         criticite   = attrs.get("criticite", "HAUTE")
76     ))
77
78     # -- Table 3 : consolidated_execution_log --
79     log_rows.append(Row(
80         run_id      = run_id,
81         query_id    = qid,
82         run_date    = run_date,
83         table       = dataset,
84         domain      = attrs.get("domaine", "Tiers"),
85         rule        = check.get("name", ""),
86         check_execution_status = "SUCCESS" if exec_ok else "
ERROR",
87         check_error_message    = "" if exec_ok else str(check
.get("error", "")),
88         check_timestamp        = now,
89         run_execution_status   = "SUCCESS",
90         run_error_message      = "",
91         updated_at             = now
92     ))
93
94     # --- 4. APPEND dans les 3 tables Hive physiques ---
95     spark.createDataFrame(history_rows).write.mode("append") \
96         .saveAsTable("dq_results.consolidated_all_column_history
")
97
98     spark.createDataFrame(config_rows).write.mode("append") \
99         .saveAsTable("dq_results.consolidated_all_checks_config"
)
100
101     spark.createDataFrame(log_rows).write.mode("append") \
102         .saveAsTable("dq_results.consolidated_execution_log")
103
104     spark.stop()
105     print(f"[DQ OK] Run {run_id} | {len(history_rows)} checks
persistes.")
106
107 if __name__ == "__main__":
108     parser = argparse.ArgumentParser()
109     parser.add_argument("--run_date", default=str(date.today()))
110     args = parser.parse_args()
111     run_dq_checks(args.run_date)

```

Le mode `append` est fondamental : à chaque exécution nocturne du DAG `daily_dq_00`, une nouvelle tranche de résultats est ajoutée aux trois tables sans écraser les données historiques. Cette accumulation constitue la base temporelle qui alimente les tendances et analyses dans le dashboard Tableau.

4.3 Création des tables Hive physiques de persistance

Les trois tables de persistance, définies lors de la conception, sont créées physiquement dans la base Hive `dq_results`. Ces tables stockent l'intégralité des métriques : les scores de qualité, les définitions SQL générées, et les statuts d'exécution. L'utilisation du format ORC avec compression Snappy garantit de très hautes performances lors des requêtes analytiques ultérieures.

```
1 CREATE DATABASE IF NOT EXISTS dq_results;
2
3 -- Table 1 : Historique des scores et resultats
4 CREATE TABLE dq_results.consolidated_all_column_history (
5     run_id STRING, query_id STRING, colonne_controlee STRING,
6     score_qualite_pct DOUBLE, statut STRING, anomalies_count
7     BIGINT
8 ) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');
9
10 -- Table 2 : Configuration des checks (code SQL genere)
11 CREATE TABLE dq_results.consolidated_all_checks_config (
12     run_id STRING, query_id STRING, sql_query STRING,
13     check_name STRING, dimension STRING, criticite STRING
14 ) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');
15
16 -- Table 3 : Journal d'exécution (statut technique)
17 CREATE TABLE dq_results.consolidated_execution_log (
18     run_id STRING, query_id STRING, execution_status STRING,
19     error_message STRING, run_timestamp TIMESTAMP
20 ) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');
```

4.4 Création du DAG Airflow indépendant

Pour garantir un découplage total avec le processus d'ingestion (Sqoop), un nouveau DAG nommé `daily_dq_00` a été configuré dans Airflow. Ce DAG s'exécute quotidiennement à 00h00. Il utilise le `PythonOperator` couplé à `subprocess` pour déclencher le job d'évaluation de la qualité. En cas d'échec d'une règle critique, l'orchestrateur remonte une erreur globale.

```
1 from airflow.operators.python import PythonOperator
2 import subprocess
3
4 def run_dq_tiers(**context):
5     run_date = context['ds']
6     result = subprocess.run(
7         ['python', '/opt/airflow/scripts/dq/dq_tiers_checks.py',
8         '--run_date', run_date],
```

```

8         capture_output=True, text=True
9     )
10    if result.returncode != 0:
11        raise Exception(f"Echec DQ Tiers:\n{result.stderr}")
12
13    dq_tiers_task = PythonOperator(
14        task_id='run_dq_tiers_checks',
15        python_callable=run_dq_tiers,
16        dag=dag
17    )

```

4.5 Exposition et virtualisation sous Dremio

Une fois les trois tables physiques alimentées dans Hive (`consolidated_all_column_history`, `consolidated_all_checks_config`, et `consolidated_execution_log`), il est nécessaire de consolider ces informations pour les exposer à l'outil BI Tableau de manière claire et optimale.

Plutôt que de créer une table physique supplémentaire ou d'effectuer des jointures complexes et coûteuses dans Tableau, nous utilisons **Dremio** en tant que couche de virtualisation sémantique.

Dans Dremio, nous créons un dataset virtuel (VDS) nommé `fact_tiers_dq_consolidated`. Cette vue réalise dynamiquement la jointure sur les clés uniques `run_id` et `query_id` des trois tables sources de Hive. Le résultat est une table de faits consolidée, exposant la totalité de la vision 360° des règles DQ, sans aucune duplication physique des données. Dremio accélère les temps de réponse de cette jointure virtuelle grâce à ses réflexions matérielles (Data Reflections).

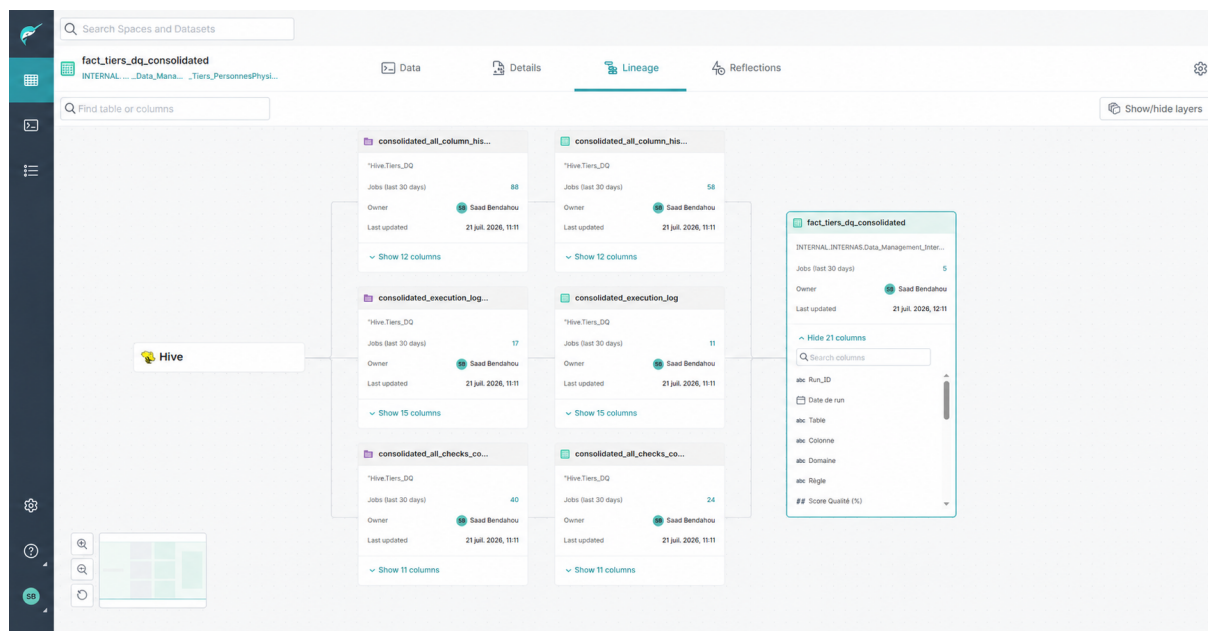


FIGURE 12 – Lignage des données dans Dremio montrant la création de la vue sémantique à partir des 3 tables Hive

4.6 Réalisation du dashboard Tableau

4.6.1 Connexion Tableau à Dremio et traitement analytique

Le dashboard Tableau est connecté à Dremio via le connecteur ODBC/JDBC natif. L'unique vue virtuelle `fact_tiers_dq_consolidated` est importée comme source de données unique dans Tableau Desktop.

Tous les filtres (par date d'exécution, par sous-domaine, par dimension de qualité), ainsi que toutes les agrégations (comptage de règles, calculs des moyennes de taux de conformité quotidiens ou scores moyens) sont réalisés de manière dynamique au sein des feuilles de calcul (sheets) et des tableaux de bord de Tableau en tirant directement les champs de la table de faits.

Note de confidentialité : Il est important de souligner que, pour des raisons évidentes de sécurité et de respect du secret bancaire, les noms de tables réels et certaines données sensibles présentées dans les captures d'écran suivantes ont fait l'objet d'un masquage ou d'un floutage systématique.

4.6.2 Onglet 1 — Vue d'ensemble (Dashboard Global)

La figure 13 présente la page d'accueil du dashboard, offrant une vision synthétique de l'état de santé du référentiel.

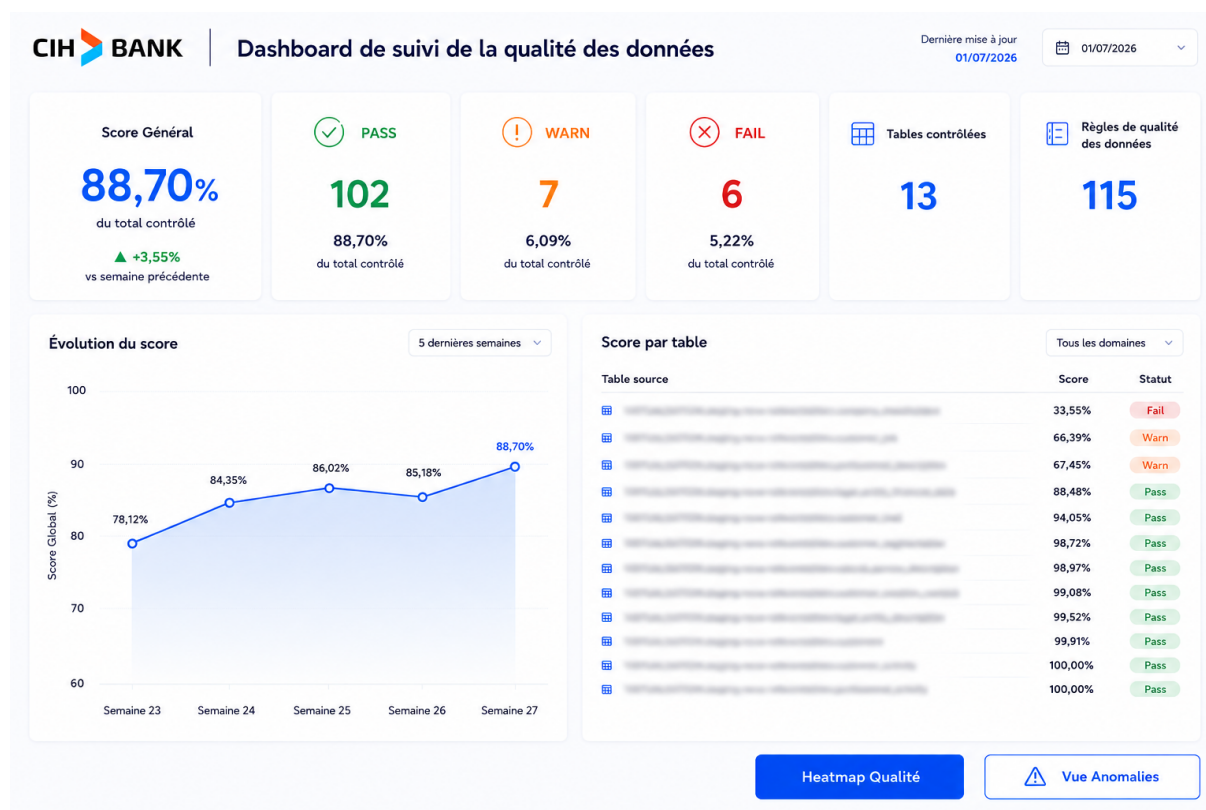


FIGURE 13 – Dashboard Tableau — Vue d'ensemble de la qualité des données

Cet onglet centralise les indicateurs de haut niveau :

- Le **Score Général** de qualité (ex : 88,70%) et sa variation par rapport à la semaine précédente.

- La répartition des contrôles par statut : le nombre de tests en succès (PASS), en avertissement (WARN) ou en échec (FAIL), accompagnés de leurs pourcentages respectifs.
- Le périmètre évalué (nombre de tables contrôlées et nombre de règles déployées).
- L'**Évolution du score** sur les 5 dernières semaines sous forme de graphique linéaire, permettant d'observer la tendance globale de la qualité.
- Un tableau de classement détaillant le **Score par table**, permettant d'identifier d'un coup d'œil les tables les plus problématiques (marquées en rouge "Fail").

4.6.3 Onglet 2 — Heatmap Qualité

La figure 14 présente la vue matricielle détaillée.

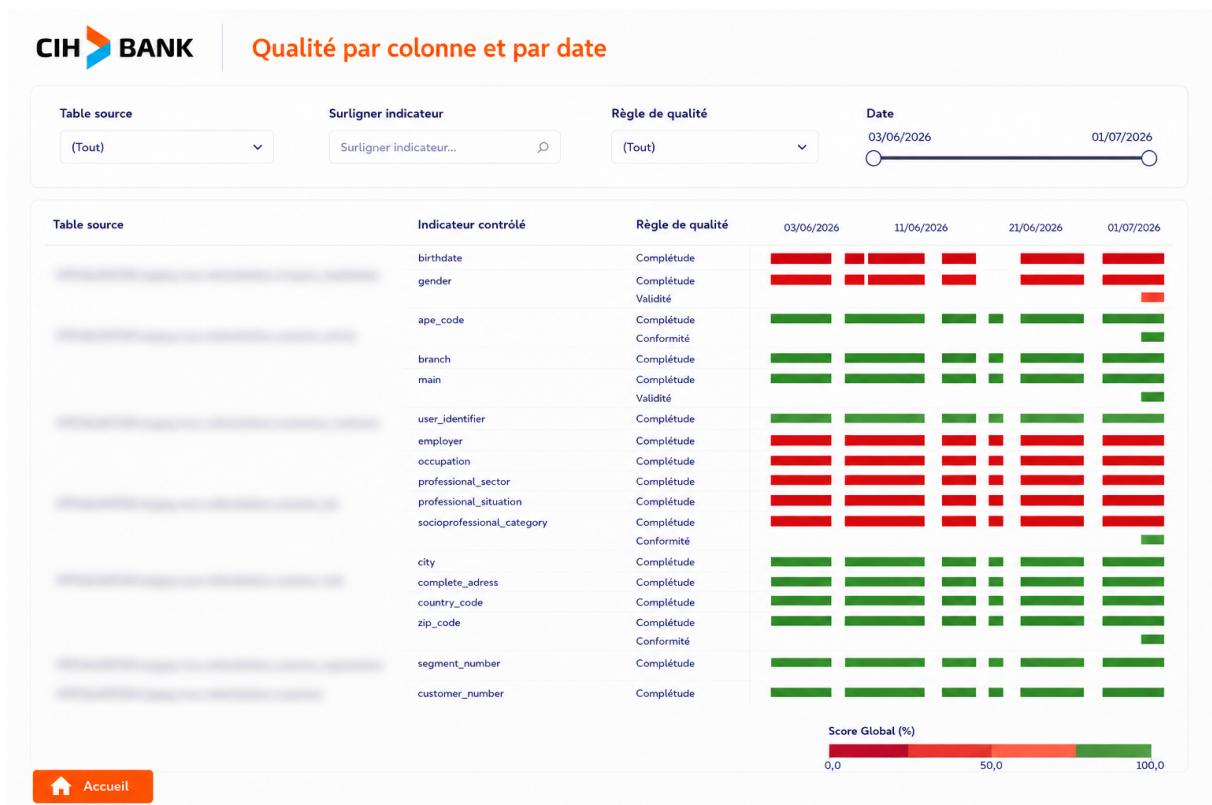


FIGURE 14 – Dashboard Tableau — Heatmap Qualité par colonne et par date

Cet onglet permet une analyse granulaire du niveau de qualité :

- Il liste précisément chaque **Indicateur contrôlé** (ex : `birthdate`, `gender`, `city`, etc.) associé à sa **Règle de qualité** (Complétude, Validité, Conformité).
- Une **carte de chaleur (Heatmap) temporelle** colore les résultats par date de scan, allant du rouge (échec total) au vert (succès total). Cette visualisation est extrêmement pertinente pour repérer rapidement des dégradations soudaines (incident de pipeline) ou confirmer l'efficacité d'une campagne de remédiation des données au fil du temps.

4.6.4 Onglet 3 — Top Anomalies

La figure 15 présente la vue orientée sur l'analyse et la remédiation.

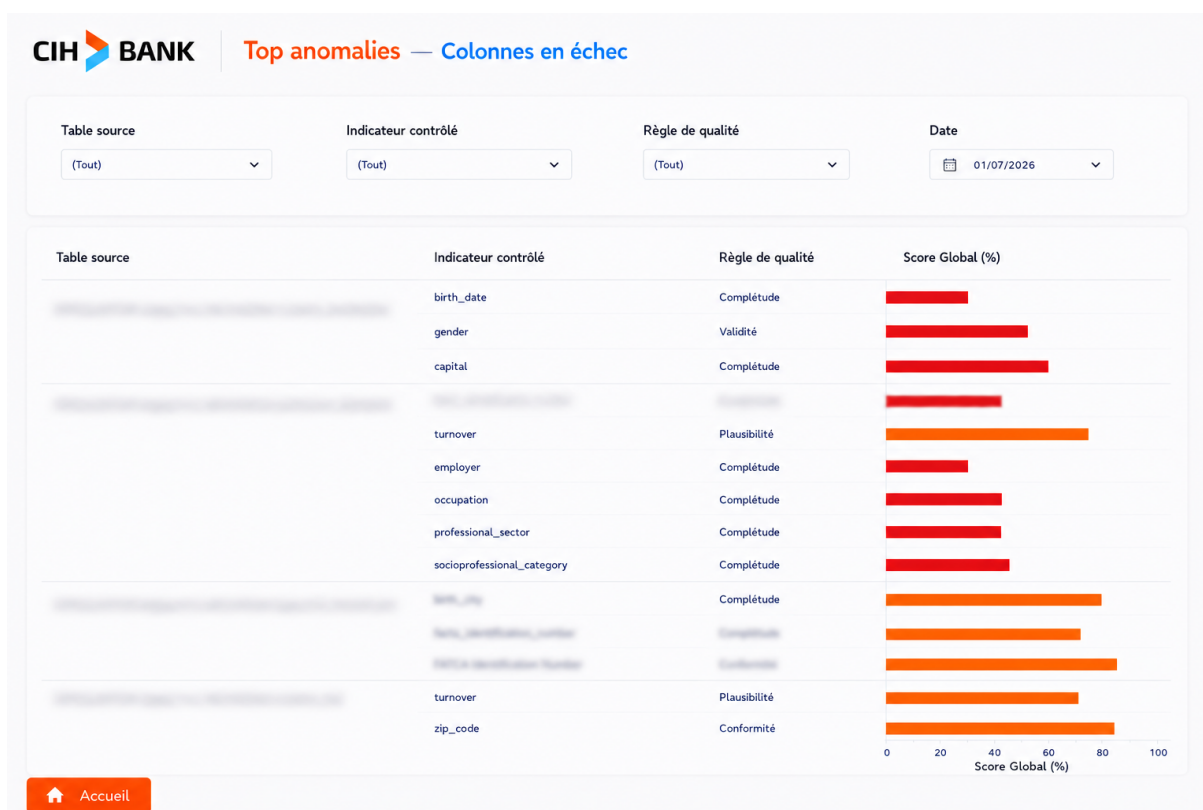


FIGURE 15 – Dashboard Tableau — Top anomalies et colonnes en échec

Ce dernier onglet est un véritable outil d'action pour les Data Stewards :

- Il cible exclusivement les règles de qualité et les colonnes présentant des taux d'anomalies élevés.
- Un diagramme en barres horizontales présente le **Score Global (%)** pour chaque indicateur en souffrance. Plus la barre s'approche du rouge et de 0, plus la volumétrie de données non-conformes est critique.
- Grâce aux filtres dynamiques situés en haut (par table, par indicateur, par règle ou par date), les utilisateurs métiers peuvent isoler instantanément les axes nécessitant des actions de correction collectives dans l'application source NOVA.

Conclusion

Ce chapitre a présenté l'implémentation concrète des différents composants du pipeline de Data Quality Tiers : le script Soda Core avec ses fichiers de configuration YAML, les tables Hive de persistance, la tâche Airflow post-ingestion, l'exposition sémantique de Dremio et le dashboard Tableau de visualisation. L'ensemble de ces composants forme un pipeline automatisé, opérationnel chaque nuit à 22h, qui permet de mesurer, historiser et restituer la qualité des données métier du Référentiel Tiers. Le chapitre suivant présente les résultats obtenus lors des premières exécutions.

Conclusion générale

Dans un écosystème bancaire en pleine mutation numérique, où la donnée constitue désormais l'infrastructure essentielle sur laquelle reposent l'évaluation des risques et la conformité réglementaire, ce Projet de Fin d'études s'est fixé pour objectif de résoudre la problématique de l'industrialisation du contrôle de la qualité des données. Le défi initial, identifié au sein de CIH Bank pour les domaines KYC et CSP, consistait à mettre un terme aux opérations de remédiation manuelles coûteuses, souvent qualifiées d'« usines à données fantômes ». L'objectif était de concevoir, d'automatiser et de déployer un pipeline complet capable de valider de bout en bout l'exactitude, la complétude, et la cohérence de l'information, tout en assurant une gouvernance proactive à la source, évitant ainsi la propagation d'erreurs dans les systèmes décisionnels en aval. Ce processus rigoureux de fiabilisation de la donnée s'inscrit également dans le strict respect de la réglementation sur la protection des données personnelles, notamment la loi 09-08 de la CNDP au Maroc et le RGPD européen, garantissant ainsi l'anonymisation et la sécurité des informations sensibles tout au long du cycle de vie analytique.

Pour répondre à ce défi, nous avons modélisé, développé et implémenté une architecture moderne de Data Lakehouse en nous appuyant sur des technologies open-source de pointe. Le moteur Soda Core, couplé à la puissance de calcul distribué d'Apache Spark, a permis de traduire des exigences métiers abstraites en requêtes analytiques performantes. L'intégration au sein d'Apache Airflow a garanti une orchestration résiliente et planifiée, tandis que l'exposition des données s'est effectuée de manière fluide via le moteur de virtualisation Dremio. Enfin, la restitution visuelle dans Tableau a transformé les résultats bruts en indicateurs décisionnels clairs et exploitables. Ce dispositif a permis d'automatiser la supervision de millions d'enregistrements, d'améliorer considérablement le temps de réconciliation, et de garantir une conformité accrue avec les exigences de traçabilité imposées par des réglementations telles que le standard BCBS 239.

Bien que le pipeline déployé ait démontré sa robustesse technique et sa valeur métier indéniable, certaines limites méritent d'être soulignées. La maintenance des fichiers YAML centralisés pourrait devenir complexe à mesure que le nombre de domaines fonctionnels couverts augmentera, et les règles implémentées restent majoritairement déterministes. Pour pallier ces limites, l'avenir du contrôle qualité s'oriente vers l'intégration de l'Intelligence Artificielle Agentique. Par exemple, face à une variable comme le genre présentant un taux de valeurs manquantes de plus de 50%, l'utilisation de modèles de langage (LLM) locaux permettrait d'inférer contextuellement l'information de manière automatisée, remplaçant un traitement manuel laborieux par un processus quasi-instantané. En dotant le pipeline de telles capacités d'observabilité intelligentes, l'institution pourra affiner dynamiquement les règles de validation et garantir que les futurs systèmes autonomes de la banque seront alimentés par des données d'une fiabilité absolue.

Bibliographie et Nétographie

Nétographie (Webographie)

1. <https://innowise.com/blog/data-analytics-in-banking/> : How data analytics is transforming the banking and financial services industry - Innowise.
2. https://scielo.org.za/scielo.php?script=sci_arttext&pid=S2304-82632024000200007 Artificial intelligence technology to enhance data quality management practices in the banking industry in South Africa.
3. <https://www.ewsolutions.com/financial-data-quality-management/> : Financial Data Quality Management : Unlocking the Power for Competitive Advantage.
4. <https://www.ibm.com/new/product-blog/bcbs239-compliance> : BCBS 239 compliance : Key findings, failures and fixes with watsonx.data intelligence - IBM.
5. https://www.researchgate.net/publication/389700967_Assessing_Data_Quality_Challenges_on_International_Banking_In_Case_of_Commercial_Bank_of_Ethiopia : (PDF) Assessing Data Quality Challenges on International Banking : In Case of Commercial Bank of Ethiopia - ResearchGate.
6. <https://corp-im.com/cost-of-bad-data-decisions/> : Cost of Bad Data Decisions : Hidden Financial Damage (2026) - Corpim.
7. https://www.researchgate.net/publication/282829737_Data_quality_in_banking_regulatory_requirements_and_best_practices : Data quality in banking : Regulatory requirements and best practices | Request PDF.
8. <https://sbs-software.com/insights/artificial-intelligence-data/data-driven-transformation-banking/> : Data-driven transformation in European banking - SBS Software.
9. <https://www.actian.com/blog/data-management/the-costly-consequences-of-poor-data-quality/> : The Consequences of Poor Data Quality : Uncovering the Hidden Risks - Actian Corporation.
10. <https://open.metu.edu.tr/handle/11511/25576> : Data quality assessment in credit risk management by customized total data quality management approach - OpenMETU.
11. <https://etd.lib.metu.edu.tr/upload/12619894/index.pdf> : data quality assessment in credit risk management by a.
12. <https://www.collibra.com/blog/the-6-dimensions-of-data-quality> : The 6 Data Quality Dimensions with Examples - Collibra.
13. <https://www.integrate.io/blog/data-quality-improvement-stats-from-etl/> : Data Quality Improvement Stats from ETL – 50+ Key Facts Every Data Leader Should Know in 2026 | Integrate.io.
14. <https://eminence.ch/en/cost-poor-data-quality/> : The \$12.9M Drain : The Real Cost of Poor Data Quality - agence Eminence.

15. <https://www.doubletrack.com/post/hidden-cost-dirty-data> : The Hidden Cost of Dirty Data : \$617B, Per Our 2026 Data Report.
16. <https://veridion.com/blog-posts/banking-capital-markets-data-strategy-development/> : How to Develop a Data Strategy in Banking and Capital Markets - Veridion.
17. <https://www.mdpi.com/1424-8220/25/14/4345> : Quantifying the Multidimensional Impact of Cyber Attacks in Digital Financial Services : A Systematic Literature Review - MDPI.
18. <https://www.bis.org/publ/bcbs239.pdf> : Basel Committee on Banking Supervision – Principles for effective risk data aggregation and risk reporting - Bank for International Settlements.
19. <https://montecarlo.ai/blog-bcbs-239-data-quality/> : BCBS 239 Data Quality Requirements : How Banks Can Ensure Compliance.
20. <https://www.solidatus.com/bcbs-239/> : What is BCBS 239? A Summary of Key Principles & Compliance - Solidatus.
21. <https://www.fluxforce.ai/regulations/bcbs-bcbs-239-risk-data-aggregation/> : BCBS 239 : Requirements, Scope, Penalties & Compliance - FluxForce AI.
22. <https://www.collibra.com/blog/how-to-achieve-data-quality-excellence-for-bcbs-239-risk-data-aggregation-compliance> : BCBS 239 data quality excellence for risk compliance - Collibra.
23. https://www.ey.com/en_nl/industries/banking-capital-markets/why-bcbs-239-compliance-is-essential-in-2025 : Why BCBS 239 Compliance is essential in 2025 | EY - Netherlands.
24. <https://soda.io/use-cases/bcbs239-compliance> : BCBS 239 Compliance with Soda Data Quality.
25. https://iason-onigiri-prod.s3.eu-south-1.amazonaws.com/Credit_Risk_Basel_IV_Regulatory_Framework_and_New_Frontiers_8db63aced7.pdf : Credit Risk : Basel IV Regulatory Framework and New Frontiers - AWS.
26. <https://kpmg.com/xx/en/our-insights/ecb-office/ecb-ratchets-up-the-pressure-on-risk-data-aggregation.html> : ECB ratchets up the pressure on risk data aggregation - KPMG International.
27. <https://www.oliverwyman.com/our-expertise/insights/2024/jul/data-management-risk-reporting-challenges-eu-banks-bcbs239.html> : Data Management And Risk Reporting Challenges For EU Banks - Oliver Wyman.
28. <https://www.grantthornton.com.cy/insights/quantitative-risk-articles/ECB-Guide-on-effective-risk-data-aggregation-and-risk-reporting/> : ECB Guide on effective risk data aggregation and risk reporting - Grant Thornton Cyprus.
29. <https://validio.io/blog/bcbs-239-from-regulatory-burden-to-data-quality> : BCBS 239 : Turning regulatory burden into data quality excellence - Validio.
30. <https://www.bairdholm.com/blog/citigroup-fined-136-million-totaling-536-million-since-2020/> : Citigroup Fined \$136 Million, Totaling \$536 Million Since 2020 | Baird Holm LLP.
31. <https://www.bankingsupervision.europa.eu/press/pr/date/2026/html/ssm.pr260219~93d5b6f73e.sv.html> : ECB sanctions J.P. Morgan for misreporting capital requirements - Banking supervision.

32. <https://basecapanalytics.com/the-real-cost-of-bad-data/> : Citigroup fined for bad loan data - Basecap Analytics.
33. <https://www.banque-france.fr/en/press-release/ecb-sanctions-bofa-securities-europe-sa-breaching-reporting-requirements> : ECB sanctions BofA Securities Europe SA for breaching reporting requirements.
34. <https://www.federalreserve.gov/newsevents/pressreleases/enforcement20240710a.htm> : Federal Reserve Board fines Citigroup \$60.6 million for violating the Board's 2020 enforcement action.
35. <https://www.experian.co.uk/blogs/latest-thinking/data-quality/hidden-cost-of-untrusted-data/> : The hidden cost of untrusted data at AI scale - Experian UK.
36. <https://store.ectap.ro/articole/699.pdf> : Credit Risk Assessment under Basel Accords - Theoretical and Applied Economics.
37. <https://www.ibm.com/think/insights/cost-of-poor-data-quality> : The True Cost of Poor Data Quality - IBM.
38. <https://www.querysurge.com/resource-center/white-papers/the-data-validation-deficit-analyzing-banking-pain-points-and-the-quest-for-effective-solutions> : White Papers - Analyzing Banking Pain Points and the Quest for Effective Solutions.
39. <https://journals.co.za/doi/abs/10.7553/90-2-2392> : Artificial intelligence technology to enhance data quality management practices in the banking industry in South Africa - Sabinet African Journals.
40. https://ideas.repec.org/p/bdi/opques/qef_666_22.html : A decision-making rule to detect insufficient data quality : an application of statistical learning techniques to the non-performing loans banking data ? - IDEAS/RePEc.
41. <https://barrmoses.medium.com/citigroup-fined-136m-for-bad-data-what-can-we-learn-1f15f82ea928> : Citigroup Fined \$136M for Bad Data. What Can We Learn ? | by Barr Moses - Medium.
42. <https://docs.soda.io/soda-core/overview-main.html> : Soda Core Official Documentation - Data Quality Management.
43. <https://spark.apache.org/docs/latest/> : Apache Spark Official Documentation - Unified Engine for Large-Scale Data Analytics.
44. <https://airflow.apache.org/docs/> : Apache Airflow Official Documentation - A platform to programmatically author, schedule, and monitor workflows.
45. <https://docs.dremio.com/> : Dremio Documentation - The Easy and Open Data Lakehouse.
46. <https://docs.python.org/3/> : Python Software Foundation - Official Documentation.
47. <https://docs.cloudera.com/> : Cloudera Documentation - Enterprise Data Cloud.